

Alert Routing Agent with Mid-Dispatch Availability Shifts

Master vibecoding plan. Complete architecture, corrected data model, and six sequential build modules — each with a pre-flight checklist, a known-defects addendum, a paste-ready prompt, a diff-review checklist, and the sentences you must be able to say on camera.

Candidate	Zisan Ahmed
Deadline	End of day, Monday 17 August 2026
Environment	Cursor · Python 3.11+ · local SQLite
Division of labour	Claude writes the code. You review, test, and explain it.
Deliverables	Public GitHub repo + 3-minute video
Supersedes	All earlier drafts and their addenda

READ THIS FIRST The brief says: *"What we care about is whether you can explain and defend every decision in your walkthrough, so do not ship anything you cannot account for."* Because Claude is writing the code, that sentence is the hardest requirement in this assessment — not the engineering. Every module therefore ends with **Read the diff** and **Explain it back**. If you cannot do the second one without notes, the module is not finished, however green the tests are.

Contents

0	How to use this document in Cursor	Who does what, rules file, one module at a time
1	Executive summary & architecture vision	Why this is hard, the four invariants, execution flow
2	Tech stack & repository layout	Mandated choices, deliberate exclusions, dependencies
3	Complete state & data architecture	SQL schema, Pydantic contracts, scoring, decision matrix
4	Module 0 — project setup	Do this before pasting any prompt
5	Module 1 — registry & query ledger	Checklist · addendum · prompt · review · explain · gate
6	Module 2 — scoring, ladder & state	Checklist · addendum · prompt · review · explain · gate
7	Module 3 — executor & the commit point	Checklist · addendum · prompt · review · explain · gate
8	Module 4 — the decision matrix	Checklist · addendum · prompt · review · explain · gate
9	Module 5 — context synthesis & explanation	Checklist · addendum · prompt · review · explain · gate
10	Module 6 — CLI, API & invariant suite	Checklist · addendum · prompt · review · explain · gate
A	Demo scenarios	Four scenarios, expected matrix rows
B	Three-minute video script	Timed, shot by shot
C	README skeleton	What the reviewer reads first
D	Defence notes	Likely questions and the answers
E	Troubleshooting	Symptom → cause → fix
F	Submission checklist	Public repo, video permissions, the reply

What the assessment actually asks for

Three requirements, quoted from the brief, each mapped to the invariant that satisfies it:

REQUIREMENT	WHERE IT IS SATISFIED
Accept an alert event and query a stakeholder registry to build a ranked list of candidates by domain expertise and availability.	Module 1 (pull) and Module 2 (ranking). Domain and availability are scored on <i>separate axes</i> — §3.4.
Execute a dispatch plan, and detect mid-execution that availability has changed <i>without re-querying stakeholders you have already evaluated</i> .	Module 1 (PresenceBus push channel) and Module 3 (interrupt listener). The event carries the new state, so knowledge refreshes while the query counter does not — §1.4.
Decide whether to complete, re-route, or escalate in parallel, such that no stakeholder receives duplicates and the final recipient has full context and an explanation of why they were chosen.	Module 4 (decision matrix R1–R11) and Module 5 (context synthesis). Duplicates are blocked by a database constraint, not by a code path — §3.1.

The brief says: *"Do not stop at the minimum. The parts where you went further than we asked, the edge case you handled, the decision you made and can defend, are the parts that count most."* Section 1.5 lists the five failure modes this design handles that the brief never mentions. Each one is a defensible decision, and each one is cheap because the architecture already implies it.

0. How to use this document in Cursor

This document is designed to be read by you and by Claude. The failure mode it guards against is an agent that makes a test pass by deleting the constraint the test exists to prove — and the second failure mode, just as costly, is shipping code you cannot account for on camera.

0.1 Who does what

ROLE	RESPONSIBILITY
CLAUDE builds	Writes every module from the prompt in this document. Does not choose the architecture, does not relax a constraint, does not skip a test.
YOU review	Read the diff against the module's review checklist before accepting it. Most of these checks are a single <code>grep</code> — they take under a minute and catch the specific ways this design gets quietly broken.
YOU test	Run the gate yourself in Cursor's terminal. Green from the agent's own report is not evidence; green in your terminal, three times, is.
YOU explain	Say the module's key decision out loud, unaided, before moving on. This is the actual deliverable — the repository is evidence, the walkthrough is the assessment.

THE STANDARD TO HOLD A module is done when **you can explain why each constraint exists and what breaks without it** — not when the tests are green. Green tests you cannot account for are worse than a missing feature you can describe honestly, because the brief tells you exactly how it will be assessed: *a person reads the repository and watches the video*.

0.2 The loop

Seven modules, strictly in order. Module 0 is setup and has no prompt. Modules 1–6 each have six parts, used in this sequence:

PART	WHO	WHAT HAPPENS
Pre-flight checklist	YOU	Tick before anything is pasted. States what must already be true — previous gate green, database reset, right files present.
Addendum	YOU	Read <i>before</i> the prompt, not after it fails. The specific defects and traps in this module. Roughly half cause failures that look like logic bugs but are not. This is also your review brief — it tells you what to look for.
The prompt	CLAUDE	Paste verbatim. Self-contained, with every correction already folded in — you are not asking the agent to reconcile two documents.
Read the diff	YOU	Targeted checks, mostly one-line greps, against the exact places this module can be silently broken. Do this <i>before</i> running the tests — a passing suite makes you credulous.
Verify & commit	YOU	Run the gate in your own terminal, inspect the database by hand, commit.
Explain it back	YOU	Two or three sentences, out loud, without looking. If you cannot, re-read the addendum and the diff until you can.

THE ONE RULE **Never let a constraint be relaxed to make a test pass.** The composite primary key on `evaluations` and the unique index on `dispatch_attempts.idempotency_key` are not implementation details — they *are* two of the four invariants the assessment is testing. If a write fails against them, the calling code is wrong. An agent's most common "fix" is to drop the constraint or make the column nullable; that turns the suite green and the submission hollow. **If a gate goes green immediately after a schema edit, read the diff before moving on.**

0.3 Put this document inside the repo

The prompts name components rather than describing them — they assume Sections 1–3 are readable in context. Keep the plan where the agent can open it:

```
lec-alert-routing-agent/  
├─ docs/  
│   └─ Master_Vibecoding_Plan_Alert_Routing_Agent.pdf    ← this document  
├─ .cursorrules      ← §0.4, write it before the first prompt  
└─ ...
```

0.4 The rules file

Create `.cursorrules` at the repository root **before Module 1**. This is the guardrail that survives every prompt:

This repo implements docs/Master_Vibecoding_Plan_Alert_Routing_Agent.pdf. Read it before changing anything.

Build strictly module by module. Do not implement a later module early.

HARD RULES – these are the assessment, not implementation details:

1. Never remove, relax, or make nullable a database constraint in order to make a test pass. PRIMARY KEY (alert_id, stakeholder_id) on evaluations IS invariant I3. UNIQUE on dispatch_attempts.idempotency_key IS invariant I2. If a write fails against either, the calling code is wrong, not the schema.
2. Never add an availability or presence term to `qualification`. Availability appears only as the separate `reachability` field. This is invariant I4.
3. Never re-query a stakeholder's availability after the initial pull. New knowledge arrives only as a PresenceBus event, applied via patch_from_push.
4. Never widen `except IntegrityError` to a bare `except Exception`, and never silence an exception to make a test pass.
5. Every Pydantic model listed in the plan's Section 3.2 stays frozen=True.
6. No numeric constant may live outside alert_router/config.py.
7. Do not add Celery, LangGraph, Redis, Alembic, or any task queue. The asyncio state machine is the deliverable, not an accident.
8. The LLM renders prose only. It never selects a recipient, never ranks, and is never given routing authority.
9. Comment WHY, not what – especially on constraints and on any line that exists to preserve an invariant.
10. If a test is hard to pass, say so and explain the conflict. Do not weaken the test.

0.5 Working practice

- ☐ **Turn off auto-accept.** Every diff gets read, and diffs touching `models_orm.py`, `db.py` or `decisions.py` get read twice.
- ☐ **Review before you run the tests.** A green suite makes you credulous — you stop looking because it already works. The review checklists are deliberately placed before the gate for this reason.
- ☐ **Ask Claude to explain, not to reassure.** "Why is this constraint here and what breaks without it?" is useful. "Does this look right?" is not — you will be told yes.

- ☐ **If you cannot follow a piece of generated code, that is a finding.** Ask for it to be simplified or commented until you can. You have to defend it on camera; incomprehensible code that works is a liability, not an asset.
- ☐ One git commit per module, with the gate result in the message. A bad edit is then one `git revert` away, and the history becomes evidence of incremental work.
- ☐ After any change touching the schema, re-run `sqlite3 alert_router.db ".schema evaluations"` and read it yourself.
- ☐ When a gate fails, fix it inside that module. Never carry a broken invariant forward — by Module 5 you will not be able to tell which layer introduced it.
- ☐ Keep the venv activated. A "module not found" from nowhere is almost always a lost venv.

THE TELL If a gate goes green *immediately after* a change to `models_orm.py`, assume a constraint was removed until you have proved otherwise. That single habit protects the two invariants that make this submission unusual.

0.6 Suggested timeline

Written Friday 14 August, due end of Monday 17 August. The modules are sized so that a failed gate does not cost you the submission.

WHEN	MODULES	NOTES
Fri evening	Module 0 + Module 1	Setup is 30 minutes. Module 1 is the foundation; do not rush the schema.
Sat	Modules 2 and 3	Module 3 is the hardest — the commit point and the abort window. Budget most of the day.
Sun	Modules 4 and 5	Module 4 is mechanical if Module 3 is correct. Module 5 is small.
Mon morning	Module 6	CLI, API, full invariant suite, README.
Mon afternoon	Record, verify, send	Record the video twice; the second take is always better. Then Appendix F — check both links in a logged-out window before replying.

IF YOU FALL BEHIND Ship Modules 1–4 with a working `reroute` and `floor` scenario and an honest README, rather than six half-finished modules. The brief is explicit: *"something small that genuinely works"* beats *"something large that does not"*. Module 5's LLM layer and Module 6's HTTP API are the two most droppable things here — the deterministic template renderer and the CLI already satisfy the brief.

1. Executive Summary & Architecture Vision

An alert fires. The agent picks the right person, starts sending, and the world changes while the message is in the air. Everything interesting in this assessment lives in that gap — and the gap only exists because sending takes time.

1.1 What actually makes this hard

The brief names three constraints that look independent and are not. Taken together they form a trap, and noticing the trap is most of the work:

1. **"Detect mid-flight that availability has changed"** — you must notice a change.
2. **"Without re-querying stakeholders you have already evaluated"** — you may not look.

You cannot notice a change you have forbidden yourself to look for. A design built on polling cannot satisfy both, and a design that quietly re-reads a cache is not satisfying the first one at all — it is just re-reading a stale answer.

THE RESOLUTION The registry exposes **two verbs, not one**. A **pull** is a question: counted, latency-bearing, and permitted exactly once per person per alert. A **push** is an announcement: free, event-driven, and it *carries the new state in its payload*. The look comes to you. Knowledge refreshes while the query counter stays still — and that is not a loophole, it is how real presence systems work. Slack and PagerDuty deliver presence as a webhook, not as a poll.

The third constraint is subtler. *"Must not escalate to someone less qualified just because they are online"* is a warning about a specific bug: if availability contributes to a candidate's score, then anyone going offline mechanically demotes them, and the next person up is whoever happens to be at their desk. The fix is structural rather than careful — see §3.4.

1.2 The four invariants

Everything below is designed around four properties that must hold on every execution path. Two are enforced by the database, one by the type system, one by arithmetic. None is enforced by a code review or a good intention.

I1 — Context preservation

A re-route never loses the alert. Every model that carries context is `frozen=True`, so nothing downstream can mutate it; the final envelope carries the *original* `AlertEvent` by reference along with the complete audit trail. **Mechanism:** immutable Pydantic models + an append-only `audit_events` table. **Test:** `test_context_survives_two_reroutes`.

I2 — No duplicate notification

Nobody is notified twice for one alert, on any channel, under any interleaving. **Mechanism:** `UNIQUE` on `dispatch_attempts.idempotency_key`, where the key is `{alert_id}:{stakeholder_id}` and *deliberately excludes the channel* — a failover to the same person is the same notification down a different pipe, so it must collide with itself. **Test:** `test_no_duplicate_across_all_scenarios`.

I3 — One availability query per person per alert

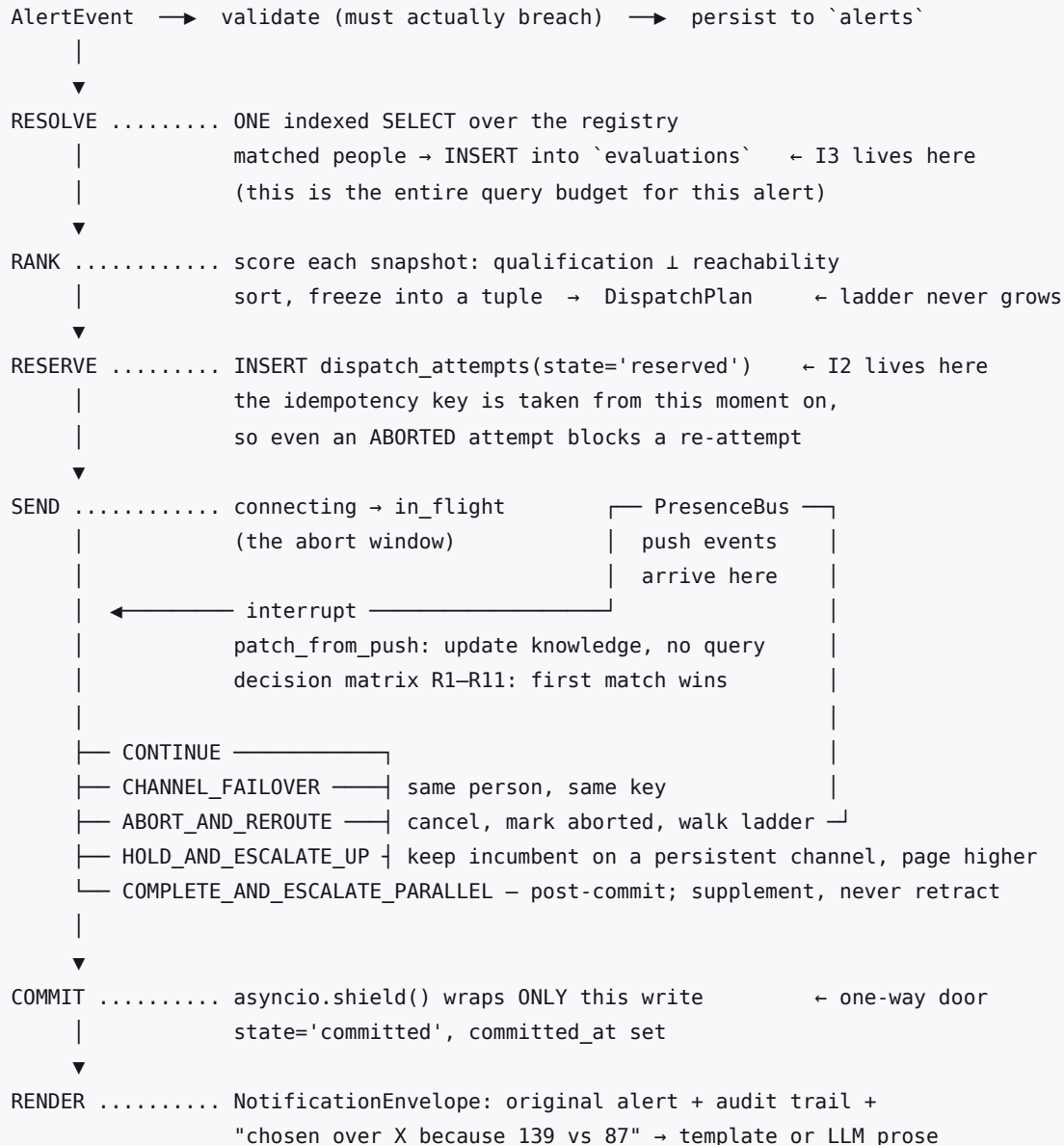
The query budget is per alert, not global — two concurrent alerts are each entitled to their own single look at the same person. **Mechanism:** `PRIMARY KEY (alert_id, stakeholder_id)` on `evaluations`. A second pull is an `IntegrityError`. **Test:** `test_second_pull_raises`.

I4 — No downward escalation

Being online never makes anyone more qualified. **Mechanism:** `qualification` contains no availability term at all, and a replacement must clear `max(MIN_QUALIFICATION[severity], incumbent.qualification - DOWNGRADE_TOLERANCE)`. **Test:** `test_offline_director_outranks_online_junior`.

Each invariant names its mechanism and its test. That pairing is the single most useful thing in the README — it converts "I was careful" into "here is the line that makes it impossible."

1.3 Core execution flow



1.4 How the mid-flight mechanism actually works

This is the paragraph to be able to say out loud in the video. Four facts, in order:

1. **The snapshot is a point-in-time observation, not a view.** When the pull returns, it returns a frozen `CandidateSnapshot` holding the status that was true at `observed_at`. It does not track the database. If the registry changes a millisecond later, the snapshot still says what it said.

2. **That staleness is the feature.** A divergence can now exist between what the agent believes and what is true. If snapshots silently refreshed, there would be no divergence — and therefore no change left to detect, and nothing for an event to tell us.
3. **The push event carries the new state.** An `InterruptEvent` has `previous` and `current` in its payload. `patch_from_push()` overwrites the cached snapshot from that payload and marks it `source="push"`. No question was asked. The `evaluations` table is untouched.
4. **The ladder is frozen; only knowledge moves.** Re-scoring from patched observations is legal, because it uses facts already paid for. *Re-resolving* — adding a person who was never in the ladder — would require a new pull, so it is forbidden. This distinction is the whole reason the system can react without spending query budget.

IN ONE SENTENCE The query counter measures *questions asked*, not *facts known*. Push events increase the second without touching the first.

1.5 Failure modes handled beyond the brief

The brief invites going further. These five cost almost nothing once the architecture is right, and each is a defensible decision in the walkthrough:

CASE	HANDLING
Depletion breaches	Stock levels and contract expiry breach <i>below</i> a threshold. A naive <code>value > threshold</code> router silently drops them. <code>direction</code> is a first-class field with a validator that refuses non-breaching alerts outright.
Channel $\neq$ person	A Slack outage is not the same event as a human going offline, and they fail independently. <code>channel_health</code> is a separate table; row R6 fails over the transport without changing the person, because changing person is an over-reaction to a transport fault.
Persistent channels	SMS and email wait on the device; Slack does not. A presence drop mid-SMS is therefore a <i>non-event</i> (row R2). This alone prevents a whole class of pointless re-routes, and it is the kind of distinction that only shows up if you think about what "offline" means.
The post-commit truth	Once a message exists you cannot unsend it. After the commit point the only honest action is a supplement, never a retraction — row R5. The abort window is explicitly bounded, and <code>asyncio.shield()</code> marks where it closes.
Loud exhaustion	If the ladder empties with nothing committed, the alert terminates as <code>exhausted</code> , recorded and visible. A silent drop is the worst possible outcome for an alerting system, so it is the one case that must never be quiet.

1.6 Explicitly out of scope

Naming these earns credit; pretending they are handled loses it. They go in the README under "What is unfinished", stated plainly:

- **Crash recovery.** A process death between `reserved` and `committed` leaves an orphaned row. Recovery would mean a sweeper that ages reserved rows out — a real design, deliberately not built.
- **Multi-process.** The `PresenceBus` is in-process. Horizontally this becomes Redis pub/sub and Postgres with `SKIP LOCKED`; the interfaces are already shaped for it.
- **Acknowledgement and de-escalation.** Nobody acks, so nothing stands down.
- **Real transports.** Channel adapters are simulated with controllable latency and failure injection, which is what makes the scenarios deterministic.
- **Time-zone-aware rotas.** `on_call` is a boolean, not a schedule.

2. Tech Stack & Repository Layout

2.1 Mandated choices

LAYER	CHOICE	WHY THIS AND NOT SOMETHING ELSE
Language	Python 3.11+	<code>asyncio.TaskGroup</code> and <code>except*</code> are 3.11 features, and the executor uses both.
Persistence	SQLite + SQLAlchemy 2.0 async (<code>aiosqlite</code>)	The brief permits "JSON or a local database". A database is chosen because <i>two invariants become schema constraints</i> — that is a materially stronger claim than "my code checks for it".
Orchestration	Custom <code>asyncio</code> state machine	The mid-flight abort window <i>is</i> the assessment. Delegating it to a framework delegates the deliverable.
Contracts	Pydantic v2, <code>frozen=True</code>	Immutability is invariant I1 expressed as a type, not as discipline.
Explanation	OpenAI API — rendering only	The LLM writes prose about a decision already made deterministically. It never routes. A template fallback means no key is ever required.
Interface	Typer + Rich (CLI), FastAPI (HTTP)	The CLI is what the video records. The HTTP surface exists so presence can be injected externally, proving the push path is real.
Testing	pytest + pytest-asyncio + freezegun	Every constraint here is a race condition. A race condition that is not tested is not handled — it is merely unobserved.

2.2 Deliberate exclusions

Worth being able to defend, because a reviewer will ask why there is no queue.

NOT USED	REASON
Celery / RQ / Arq	A broker moves work to another process, which puts the cancellation window out of reach. The whole point is to abort a dispatch that is already in flight <i>in this process</i> .
LangGraph / agent frameworks	The decision logic is a deterministic truth table (§3.5), not a reasoning loop. Wrapping eleven predicates in an LLM graph would make the system slower, non-deterministic, and impossible to test exhaustively.
Alembic	Single-shot schema creation. Migrations would be ceremony for a three-day build with no production data.
Redis	In-process pub/sub is sufficient and honest about its limits. Named in "what is unfinished" rather than half-built.

2.3 Repository layout

```

lec-alert-routing-agent/
├── README.md                # run instructions, invariants, what is unfinished
├── pyproject.toml           # incl. [tool.pytest.ini_options] – see §2.4
├── requirements.txt
├── .env.example             # OPENAI_API_KEY= (empty; optional)
├── .gitignore               # .env, *.db*, .venv – write BEFORE first commit
├── .cursorrules             # §0.4
├── docs/
│   └── Master_Vibecoding_Plan_Alert_Routing_Agent.pdf
├── data/
│   └── seed_stakeholders.json # 10 people → SQLite on first run
├── alert_router/
│   ├── __init__.py
│   ├── config.py           # ALL tunables: weights, floors, latency, seed [Module 1]
│   ├── db.py               # async engine, PRAGMA listener, init_db, seed [Module 1]
│   ├── models_orm.py       # SQLAlchemy 2.0 tables – the constraints [Module 1]
│   ├── schemas.py          # Pydantic v2 frozen contracts [Module 1]
│   ├── registry.py         # PULL (counted) + PUSH (PresenceBus) [Module 1]
│   ├── ranking.py          # scoring, ladder construction, clears_floor [Module 2]
│   ├── state.py            # DispatchState: ledgers, lock, audit [Module 2]
│   ├── channels.py         # adapters + persistence classification [Module 3]
│   ├── executor.py         # two-phase send, the commit point, the shield [Module 3]
│   ├── interrupts.py       # listener, relevance filter, BETTER_MATCH [Module 3]
│   ├── decisions.py        # the decision matrix R1–R11 [Module 4]
│   ├── context.py          # context compiler + OpenAI renderer [Module 5]
│   ├── agent.py            # top-level orchestration (TaskGroup) [Module 4]
│   ├── api.py              # FastAPI: POST /alerts, POST /presence [Module 6]
│   └── cli.py              # Typer + Rich demo [Module 6]
├── tests/
│   ├── conftest.py         # engine fixture, deterministic clock [Module 1]
│   ├── test_registry.py    # I3 at the data layer [Module 1]
│   ├── test_ranking.py     # I4 – qualification ⊥ availability [Module 2]
│   ├── test_executor.py    # commit point, abort window, the shield [Module 3]
│   ├── test_decisions.py   # the full truth table, parametrised [Module 4]
│   ├── test_context.py     # I1 across two re-routes [Module 5]
│   └── test_invariants.py  # all four, end to end, every scenario [Module 6]

```

2.4 Dependencies and test configuration

requirements.txt :

```

fastapi>=0.110      sqlalchemy>=2.0      aiosqlite>=0.19
uvicorn[standard]>=0.27 pydantic>=2.6      pydantic-settings>=2.2
typer>=0.12         rich>=13.7          openai>=1.14
pytest>=8.0         pytest-asyncio>=0.24 httpx>=0.27
freezegun>=1.4

```

PIN CORRECTION `pytest-asyncio` is pinned to **≥0.24**, not ≥0.23. Versions 0.23.0–0.23.5 shipped event-loop-scope regressions around async fixtures that produce failures indistinguishable from application bugs. You will lose an hour to this if you take the older pin.

These four lines go in `pyproject.toml` and must be present *before* the first test is written, not added afterwards:

```

[tool.pytest.ini_options]
asyncio_mode = "auto"                # else every test needs a marker
asyncio_default_fixture_loop_scope = "function" # silences the 0.23+ deprecation
pythonpath = ["."]                   # imports alert_router without pip install -e .
testpaths = ["tests"]

```

2.5 Environment and credentials

Modules 1 through 4 require no API key, no credential, and no external service. They are SQLite on local disk. The OpenAI key first matters in Module 5, and even there a deterministic template renderer means the demo runs without one — the README's run instructions say `--scenario all` needs no key, and that is true.

PUBLIC REPO The repository must be public to be assessed. Write `.gitignore` **before the first commit**, excluding `.env`, `*.db`, `*.db-wal`, `*.db-shm`, `.venv/`, `__pycache__/`. A real `OPENAI_API_KEY` pushed to a public repository is detected by OpenAI's scanner and auto-revoked, usually within minutes. Only `.env.example`, with an empty value, is ever committed.

3. Complete State & Data Architecture

Two of the four invariants are enforced by database constraints rather than by application logic. Read the `UNIQUE` and `PRIMARY KEY` clauses below as the actual specification — the Python is downstream of them.

3.1 SQL schema

```

-- -----
-- stakeholders : the registry. Domains are ORDERED – primary_domain is
-- the person's specialism, secondary_domains is adjacent competence.
-- Splitting them is what lets ranking score a primary-domain L3 above a
-- secondary-domain L4, which is the mechanism behind I4.
-- on_call is SEPARATE from status on purpose: being rostered and being
-- at your desk are different facts, and conflating them is how you end
-- up paging whoever happens to be awake.
-- -----
CREATE TABLE stakeholders (
  id          TEXT PRIMARY KEY,
  name        TEXT NOT NULL,
  title       TEXT NOT NULL DEFAULT '',
  primary_domain TEXT NOT NULL,
  secondary_domains TEXT NOT NULL DEFAULT '[]', -- JSON array
  seniority_tier INTEGER NOT NULL CHECK (seniority_tier BETWEEN 1 AND 5),
  preferred_channel TEXT NOT NULL CHECK (preferred_channel IN ('slack','email','sms')),
  fallback_channels TEXT NOT NULL DEFAULT '[]', -- JSON array, ORDERED
  status      TEXT NOT NULL CHECK (status IN ('online','busy','offline')),
  on_call     INTEGER NOT NULL DEFAULT 0,
  updated_at  REAL NOT NULL
);
CREATE INDEX ix_stakeholders_primary_domain ON stakeholders(primary_domain);

-- -----
-- channel_health : models "their preferred channel is unavailable".
-- A person is not their transport; these fail independently.
-- SEEDDED at init for every (person, channel) in their channel_order, so
-- "is there a healthy fallback?" is a plain SELECT rather than a query
-- that has to treat a missing row as implicitly healthy.
-- -----
CREATE TABLE channel_health (
  stakeholder_id TEXT NOT NULL REFERENCES stakeholders(id),
  channel         TEXT NOT NULL CHECK (channel IN ('slack','email','sms')),
  healthy        INTEGER NOT NULL DEFAULT 1,
  last_error     TEXT,
  updated_at     REAL NOT NULL,
  PRIMARY KEY (stakeholder_id, channel)
);

-- -----
-- alerts : immutable once written except for `state`. direction supports
-- depletion metrics (stock, contract expiry) that breach BELOW.
-- NOTE: this row MUST exist before any evaluation references it – see
-- ensure_alert() in Module 1. With PRAGMA foreign_keys=ON the first
-- INSERT into evaluations fails otherwise.
-- -----
CREATE TABLE alerts (
  alert_id      TEXT PRIMARY KEY,
  metric_name   TEXT NOT NULL,
  value        REAL NOT NULL,
  threshold     REAL NOT NULL,
  direction     TEXT NOT NULL CHECK (direction IN ('above','below')),
  severity      TEXT NOT NULL CHECK (severity IN ('low','high','critical')),
  domain       TEXT NOT NULL,
  source        TEXT NOT NULL DEFAULT 'metrics-pipeline',
  triggered_at  REAL NOT NULL,
  state         TEXT NOT NULL DEFAULT 'routing'
  CHECK (state IN ('routing','delivered','escalated','exhausted'))
);

-- -----
-- evaluations : THE QUERY LEDGER. Invariant I3 lives in this PRIMARY KEY.
-- A second availability pull for the same (alert, person) is an
-- IntegrityError – the constraint cannot be forgotten or bypassed.

```

```

--
-- qualification and ladder_rank are written as SENTINELS (0.0 and -1) at
-- pull time, because scoring belongs to Module 2 and putting it in the
-- registry would collapse the layer split. build_ladder() updates them.
-- -1 must never be read as "rank 1".
--
--
-- `source` is 'pull' for every row this system ever writes. Push events
-- deliberately do NOT touch this table – that is what makes the ledger a
-- true count of questions asked. The 'push' value exists so the column
-- can express provenance if a future design ever needs it.
--
CREATE TABLE evaluations (
    alert_id          TEXT NOT NULL REFERENCES alerts(alert_id),
    stakeholder_id    TEXT NOT NULL REFERENCES stakeholders(id),
    observed_status    TEXT NOT NULL,
    observed_at        REAL NOT NULL,
    qualification      REAL NOT NULL DEFAULT 0.0,
    reachability       INTEGER NOT NULL,
    ladder_rank        INTEGER NOT NULL DEFAULT -1,
    source             TEXT NOT NULL CHECK (source IN ('pull','push')),
    PRIMARY KEY (alert_id, stakeholder_id)
);

--
--
-- dispatch_attempts : THE DUPLICATE KILLER. Invariant I2 lives in the
-- UNIQUE idempotency_key. The key EXCLUDES channel: a failover to the
-- same person is the SAME notification down a different pipe, so it must
-- collide with itself and be handled as a continuation.
-- The row is inserted at RESERVE, before sending – which is precisely why
-- an ABORTED attempt still blocks a later re-attempt on the same person.
--
CREATE TABLE dispatch_attempts (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    alert_id          TEXT NOT NULL REFERENCES alerts(alert_id),
    stakeholder_id    TEXT NOT NULL REFERENCES stakeholders(id),
    idempotency_key    TEXT NOT NULL UNIQUE,          -- '{alert_id}:{stakeholder_id}'
    channel           TEXT NOT NULL,
    role              TEXT NOT NULL CHECK (role IN ('primary','reroute','escalation','fyi')),
    state             TEXT NOT NULL CHECK (state IN
        ('reserved','connecting','in_flight','committed','aborted','failed')),
    plan_version       INTEGER NOT NULL DEFAULT 1,
    reserved_at        REAL NOT NULL,
    committed_at      REAL,
    outcome_reason     TEXT
);

--
--
-- audit_events : append-only. This IS the context preserved across
-- re-routes (I1) and rendered into the "why you" explanation.
-- No UPDATE or DELETE is ever issued against this table.
-- `seq` is allocated under the DispatchState lock so it stays monotonic
-- when the executor and the interrupt listener both write.
--
CREATE TABLE audit_events (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    alert_id          TEXT NOT NULL REFERENCES alerts(alert_id),
    seq              INTEGER NOT NULL,
    at               REAL NOT NULL,
    kind             TEXT NOT NULL,          -- RESOLVED | RANKED | RESERVED | INTERRUPT |
                                           -- DECISION | ABORTED | COMMITTED | SUPPRESSED
    actor            TEXT NOT NULL,          -- stakeholder_id or subsystem name
    summary          TEXT NOT NULL,          -- human-readable, goes straight into the video
    payload          TEXT NOT NULL DEFAULT '{}')
);
CREATE UNIQUE INDEX ux_audit_alert_seq ON audit_events(alert_id, seq);

```

CONNECTION PRAGMAS `PRAGMA foreign_keys` is **per-connection**, not per-database. Setting it once inside `init_db()` leaves every subsequent pooled connection unenforced — *silently*. It must be registered as a `connect` event listener so it fires for every DBAPI connection, alongside WAL mode and a busy timeout that Module 3 will need:

```
@event.listens_for(engine.sync_engine, "connect")
def _pragmas(dbapi_conn, _):
    cur = dbapi_conn.cursor()
    cur.execute("PRAGMA foreign_keys=ON")      # or the FKs above are decorative
    cur.execute("PRAGMA journal_mode=WAL")    # concurrent read during write (Module 3)
    cur.execute("PRAGMA busy_timeout=5000")   # prevents 'database is locked' under TaskGroup
    cur.close()
```

3.2 Pydantic v2 contracts

Complete and importable as written — v1 of this plan omitted the `uuid4` and `time` imports and never defined `AttemptRecord` or `AuditEvent`, both of which `DispatchState` references.

```

from __future__ import annotations
import time
from enum import Enum
from typing import Literal
from uuid import uuid4
from pydantic import BaseModel, ConfigDict, Field, model_validator

FROZEN = ConfigDict(frozen=True)      # I1: nothing downstream can mutate context

class Severity(str, Enum):
    LOW = "low"; HIGH = "high"; CRITICAL = "critical"
    @property
    def rank(self) -> int: return {"low":1,"high":2,"critical":3}[self.value]

class Availability(str, Enum):
    ONLINE = "online"; BUSY = "busy"; OFFLINE = "offline"
    @property
    def reachability(self) -> int:      # 2/1/0 – a SEPARATE axis from qualification
        return {"online":2,"busy":1,"offline":0}[self.value]

class Channel(str, Enum):
    SLACK = "slack"; EMAIL = "email"; SMS = "sms"
    @property
    def is_persistent(self) -> bool:
        """Email/SMS survive the recipient being offline; Slack does not.
        This property is why a presence drop mid-SMS is a NON-EVENT (row R2)."""
        return self is not Channel.SLACK

# — Alert —————
class AlertEvent(BaseModel):
    model_config = FROZEN
    alert_id: str = Field(default_factory=lambda: f"alr-{uuid4().hex[:8]}")
    metric_name: str
    value: float
    threshold: float
    direction: Literal["above", "below"] = "above"
    severity: Severity
    domain: str
    source: str = "metrics-pipeline"
    triggered_at: float = Field(default_factory=time.time)

    @model_validator(mode="after")
    def _must_actually_breach(self):
        breached = (self.value > self.threshold) if self.direction == "above" \
            else (self.value < self.threshold)
        if not breached:
            raise ValueError(f"{self.metric_name}={self.value} does not breach "
                             f"{self.threshold} ({self.direction})")
        return self

    @property
    def breach_magnitude(self) -> float:
        return abs(self.value - self.threshold) / abs(self.threshold or 1)

# — Registry —————
class StakeholderRecord(BaseModel):
    model_config = FROZEN
    id: str; name: str; title: str = ""
    primary_domain: str
    secondary_domains: tuple[str, ...] = ()      # tuple, NOT list – frozen needs hashable
    seniority_tier: int = Field(ge=1, le=5)
    preferred_channel: Channel
    fallback_channels: tuple[Channel, ...] = ()
    status: Availability
    on_call: bool = False

```



```

def domain_fit(self, domain: str) -> Literal["primary", "secondary", "none"]:
    if domain == self.primary_domain: return "primary"
    return "secondary" if domain in self.secondary_domains else "none"

@property
def channel_order(self) -> tuple[Channel, ...]:
    return (self.preferred_channel, *self.fallback_channels)

# — The point-in-time observation —————
class CandidateSnapshot(BaseModel):
    """What ONE pull returned. `status` is what was TRUE AT observed_at and is
    never silently refreshed — the gap between this and reality IS the
    mid-flight signal. A push produces a NEW snapshot with source='push', so
    the audit trail records how we learned each fact."""
    model_config = FROZEN
    alert_id: str
    stakeholder: StakeholderRecord
    status: Availability
    observed_at: float
    source: Literal["pull", "push"] = "pull"

class ScoreBreakdown(BaseModel):
    """Every term recorded separately so the notification can say 'chosen over
    Marcus because primary-domain 100 vs secondary 55'."""
    model_config = FROZEN
    domain_points: float
    seniority_points: float
    on_call_points: float
    qualification: float      # sum of the three above — NO availability term
    reachability: int         # 2 | 1 | 0 — the SECOND, independent key
    eligible: bool
    notes: tuple[str, ...] = ()

class RankedCandidate(BaseModel):
    model_config = FROZEN
    snapshot: CandidateSnapshot
    score: ScoreBreakdown
    rank: int

class DispatchPlan(BaseModel):
    """Frozen at ranking time. Ladder MEMBERSHIP never grows without a new
    pull, which is what makes I3 structural rather than aspirational."""
    model_config = FROZEN
    alert: AlertEvent
    ladder: tuple[RankedCandidate, ...]
    created_at: float
    plan_version: int = 1

# — Attempts and audit (referenced by DispatchState, §3.3) —————
class AttemptState(str, Enum):
    RESERVED = "reserved"; CONNECTING = "connecting"; IN_FLIGHT = "in_flight"
    COMMITTED = "committed"; ABORTED = "aborted"; FAILED = "failed"
    @property
    def is_terminal(self) -> bool:
        return self in (AttemptState.COMMITTED, AttemptState.ABORTED, AttemptState.FAILED)
    @property
    def is_pre_commit(self) -> bool:
        return self in (AttemptState.RESERVED, AttemptState.CONNECTING,
                        AttemptState.IN_FLIGHT)

class AttemptRecord(BaseModel):
    model_config = ConfigDict(frozen=False)  # the ONE mutable record, guarded by the lock
    alert_id: str
    stakeholder_id: str
    channel: Channel

```

```

    role: Literal["primary", "reroute", "escalation", "fyi"]
    state: AttemptState = AttemptState.RESERVED
    plan_version: int = 1
    reserved_at: float
    committed_at: float | None = None
    outcome_reason: str = ""

    @property
    def idempotency_key(self) -> str:
        return f"{self.alert_id}:{self.stakeholder_id}" # channel EXCLUDED – see I2

class AuditEvent(BaseModel):
    model_config = FROZEN
    alert_id: str
    seq: int
    at: float
    kind: str
    actor: str
    summary: str
    payload: dict = Field(default_factory=dict)

# — Interrupts —————
class InterruptKind(str, Enum):
    PRESENCE_CHANGED = "presence_changed"
    CHANNEL_DEGRADED = "channel_degraded"
    BETTER_MATCH = "better_match" # DERIVED by the listener, never raw

class InterruptEvent(BaseModel):
    model_config = FROZEN
    kind: InterruptKind
    stakeholder_id: str
    previous: Availability | None = None
    current: Availability | None = None
    channel: Channel | None = None
    at: float
    reason: str = ""

    @property
    def went_offline(self) -> bool:
        return bool(self.previous and self.current
                    and self.previous.reachability > 0
                    and self.current.reachability == 0)

# — Decisions —————
class DecisionAction(str, Enum):
    CONTINUE_UNCHANGED = "continue_unchanged"
    CHANNEL_FAILOVER = "channel_failover"
    ABORT_AND_REROUTE = "abort_and_reroute"
    COMPLETE_AND_ESCALATE_PARALLEL = "complete_and_escalate_parallel"
    HOLD_AND_ESCALATE_UP = "hold_and_escalate_up"
    EXHAUSTED = "exhausted"

class RoutingDecision(BaseModel):
    model_config = FROZEN
    action: DecisionAction
    target_id: str | None = None # re-route / failover target
    escalate_to_id: str | None = None # parallel escalation target
    rationale: str # one sentence, goes in the audit trail
    matrix_row: str # 'R4' – traceable to §3.5
    suppressed: tuple[tuple[str, str], ...] = () # (stakeholder_id, why)

# — Final payload —————
class NotificationEnvelope(BaseModel):
    """Everything the recipient gets. Carries the ORIGINAL AlertEvent (not a
    copy), the full audit trail, and the comparison justifying this person over
    the runner-up. This model IS invariant I1."""

```

```

model_config = FROZEN
alert: AlertEvent
recipient: StakeholderRecord
channel: Channel
role: Literal["primary", "reroute", "escalation", "fyi"]
plan_version: int
chosen_because: str
considered_and_passed: tuple[tuple[str, str], ...] # (name, why not)
audit_trail: tuple[str, ...]
rendered_body: str # template or LLM

```

3.3 DispatchState — the mutable core

One mutable object per alert. Everything it *holds* is immutable, so there is exactly one place concurrent writes can go wrong and exactly one lock protecting it.

```

@dataclass
class DispatchState:
    alert: AlertEvent
    plan: DispatchPlan
    lock: asyncio.Lock = field(default_factory=asyncio.Lock)

    # Three DISJOINT ledgers plus suppression. Keeping them separate is what
    # lets the system answer "did we ask?", "did we try?" and "did we send?"
    # independently – collapsing them is how duplicates appear.
    evaluated: set[str] = field(default_factory=set) # I3: asked
    observed: dict[str, CandidateSnapshot] = field(default_factory=dict) # patch-on-push
    attempted: dict[str, AttemptRecord] = field(default_factory=dict) # incl. ABORTED
    notified: set[str] = field(default_factory=set) # committed only
    suppressed: dict[str, str] = field(default_factory=dict) # id → numeric why

    current_attempt: AttemptRecord | None = None
    plan_version: int = 1
    audit: list[AuditEvent] = field(default_factory=list)
    terminal: bool = False
    _seq: int = 0

    def patch_from_push(self, ev: InterruptEvent) -> bool:
        """Update knowledge WITHOUT a query. The event payload carries the new
        state, so no question is asked and the pull ledger never moves.
        Returns False for a subject we never evaluated – a cross-domain event
        about a stranger is not ours to act on (matrix row R1), and indexing
        blindly here would raise KeyError on perfectly normal traffic."""
        prev = self.observed.get(ev.stakeholder_id)
        if prev is None or ev.current is None:
            return False
        self.observed[ev.stakeholder_id] = prev.model_copy(
            update={"status": ev.current, "observed_at": ev.at, "source": "push"})
        return True

    def next_candidate(self) -> RankedCandidate | None:
        """Walk the FROZEN ladder in rank order, skipping anyone in `attempted`
        – including aborted attempts. Skipping only `notified` here would let an
        aborted person be re-attempted, and their idempotency key is already
        taken, so it would surface as an IntegrityError at the worst moment."""
        for cand in self.plan.ladder:
            sid = cand.snapshot.stakeholder.id
            if sid not in self.attempted and sid not in self.suppressed:
                return cand
        return None

```

3.4 The scoring function

Qualification deliberately contains **no availability term**. This is the mechanical expression of I4: because presence is not an addend, no amount of being online can raise a junior above a senior. Ranking then sorts on (reachability, qualification, seniority_tier) — reachable first, because an unreachable person cannot be the *initial* recipient — but every *replacement* comparison uses qualification alone.

```
DOMAIN_POINTS    = {"primary": 100.0, "secondary": 55.0, "none": None} # None → ineligible
SENIORITY_POINTS = 8.0      # × tier (1.5) → 8 ... 40
ON_CALL_POINTS   = 15.0

qualification = DOMAIN_POINTS[fit] + SENIORITY_POINTS * tier + (ON_CALL_POINTS if on_call else 0)

MIN_QUALIFICATION = {Severity.LOW: 0.0, Severity.HIGH: 90.0, Severity.CRITICAL: 120.0}
DOWNGRADE_TOLERANCE = 25.0 # ≈ one seniority step plus on-call duty

sort_key = (reachability, qualification, seniority_tier) # descending
reroute_ok = candidate.qualification >= max(MIN_QUALIFICATION[severity],
                                              incumbent.qualification - DOWNGRADE_TOLERANCE)
```

WORKED EXAMPLE — CRITICAL / INFRASTRUCTURE Priya Raman, L3 SRE, primary domain, on-call → 100 + 24 + 15 = **139**, online. Tom Beckett, L1 SRE, primary domain, on-call → 100 + 8 + 15 = **123**, online. Elena Fischer, L5 Director, primary domain, not on-call → 100 + 40 + 0 = **140**, offline. Initial pick is Priya (reachable, 139). When she drops offline the floor is $\max(120, 139 - 25) = 120$; Tom clears it at 123, so re-routing is legal. Take Tom off-call (108) and the floor rejects him — the agent holds and escalates *up* to Elena on SMS instead. Both branches are demonstrated in Appendix A.

3.5 The decision matrix

Rows are evaluated top to bottom; **the first match wins**. Implement it as a literal ordered list of (predicate, handler) pairs, not as nested `if` blocks — nesting is how R2 gets shadowed by R3 without anyone noticing. Every `RoutingDecision` carries the `matrix_row` that fired, so the audit trail is traceable to this table and the video can point at a line.

#	INTERRUPT	PHASE	CONDITION	DECISION & REASONING
R1	any	any	subject not in ladder $\wedge$ not incumbent	CONTINUE_UNCHANGED — cross-domain noise, never wakes the matrix.
R2	PRESENCE → offline	pre-commit	in-flight channel <code>is_persistent</code>	CONTINUE_UNCHANGED — SMS/email wait on the device. Offline means "away from keyboard", not "unreachable". Prevents a whole class of pointless re-routes.
R3	PRESENCE → offline	pre-commit	synchronous channel $\wedge \exists$ candidate clearing the floor	ABORT_AND_REROUTE — cancel inside the abort window, mark <code>aborted</code> , walk the frozen ladder. Zero new queries.
R4	PRESENCE → offline	pre-commit	synchronous channel $\wedge$ no candidate clears the floor	HOLD_AND_ESCALATE_UP — keep the incumbent on a persistent channel <i>and</i> page the highest-qualification member. This row is I4. The online junior is recorded in <code>suppressed</code> with the arithmetic.
R5	PRESENCE → offline	post-commit	—	COMPLETE_AND_ESCALATE_PARALLEL — the message already exists; you cannot unsend it. Honest correction is a supplement, not a retraction.
R6	CHANNEL_DEGRADED	pre-commit	channel is the one in flight $\wedge$ a healthy fallback exists	CHANNEL_FAILOVER — same person, same idempotency key, so it cannot register as a duplicate. Changing person here would over-react to a transport fault.

R7	CHANNEL_DEGRADED	pre-commit	no healthy channel remains	ABORT_AND_REROUTE — the person is genuinely unreachable; now changing person is correct.
R8	BETTER_MATCH	any	subject already in notified	CONTINUE_UNCHANGED — I2 outranks optimality. Never re-notify to improve a decision.
R9	BETTER_MATCH	any	new.qualification > incumbent.qualification $\wedge$ severity $\geq$ HIGH	COMPLETE_AND_ESCALATE_PARALLEL — original completes as fyi , senior gets the full page. Different people $\Rightarrow$ different keys $\Rightarrow$ duplicate-free by construction.
R10	BETTER_MATCH	any	severity = LOW	CONTINUE_UNCHANGED — do not wake a director for a low-severity alert that is already correctly routed. Escalation has a cost, and the cost is a human's attention.
R11	any	any	ladder exhausted, nothing committed	EXHAUSTED — terminal, loud, recorded. Visible failure beats a silent drop.

ORDERING TRAP R2 and R5 both fire on *presence* $\rightarrow$ *offline* and differ **only** by phase. If the phase predicate is missing or evaluated loosely, a post-commit drop on SMS matches R2 and the parallel escalation never happens. Test both phases for the same event explicitly.

3.6 Legal state transitions

FROM	TO	TRIGGER / NOTE
reserved	connecting , aborted	The row exists in the DB, so the idempotency key is already taken — this is why an abort still blocks a later duplicate.
connecting	in_flight , aborted , failed	Channel adapter handshake. A refused connect is failed , not aborted .
in_flight	committed , aborted , failed	Last abortable state. Cancellation raises CancelledError , which is caught, written as aborted , and then re-raised unless the abort was deliberate.
committed	— (terminal)	One-way door. Only supplements are possible after this.
aborted / failed	— (terminal for that attempt)	The stakeholder stays in attempted forever.

Project Setup

Thirty minutes. Everything here must be done before you paste a single prompt — three of these steps are irreversible-ish if skipped, and one of them protects a live API key.

STEP BY STEP

1 · VERIFY THE TOOLCHAIN

```
python3 --version          # need 3.11 or newer
python3 -c "import sqlite3; print(sqlite3.sqlite_version)" # need 3.38+
python3 -c "import sqlite3; sqlite3.connect(':memory:').execute(
    \"select value from json_each('[\\\"a\\\"]')\"); print('JSON1 ok')\"
git --version
```

If `json_each` raises, the secondary-domain query in Module 1 has no clean implementation. Stop and say so rather than falling back to `LIKE '%infra%'`, which is both injectable and wrong for overlapping domain names.

2 · CREATE THE FOLDER — EMPTY

```
mkdir -p ~/Downloads/lec-alert-routing-agent/docs
cd ~/Downloads/lec-alert-routing-agent
```

DO NOT IMPORT THE OLD PROTOTYPE Any earlier `models.py`, `registry.py`, `demo_phase1.py` or `stakeholders.json` from a previous attempt must **not** be copied in. That code used `stdlib` dataclasses and an in-memory dict, and enforced the no-double-query rule with a Python `if`. Module names collide, and you would lose the strongest answer in Appendix D — *“it is the primary key of `evaluations` ; it is not a convention I follow, it is a write the database refuses.”* Start empty.

3 · PUT THIS PLAN INSIDE THE REPO

```
docs/Master_Vibecoding_Plan_Alert_Routing_Agent.pdf
```

Inside, not beside. The prompts name components rather than describing them, so they are only meaningful with Sections 1–3 readable in context.

4 · WRITE `.CURSORRULES`

Copy the block from §0.4 verbatim to the repository root. Do this **now**, not after the first failure — its entire purpose is to be in place before the agent meets a failing constraint.

5 · WRITE `.GITIGNORE` — BEFORE ANY COMMIT

```
.venv/
__pycache__/
*.py[cod]
.env
*.db
*.db-wal
*.db-shm
.pytest_cache/
.DS_Store
```

6 · INITIALISE GIT

```
git init
git add .gitignore .cursorrules docs/
git commit -m "chore: scaffold, architecture plan, agent guardrails"
```

7 · CREATE AND ACTIVATE THE VIRTUAL ENVIRONMENT

```
python3 -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
python -m pip install --upgrade pip
```

Keep it activated for every module. In Cursor, open a terminal once and stay in it — a "module not found" that appears from nowhere is almost always a lost venv.

8 · TURN OFF AUTO-ACCEPT IN CURSOR

Your job for the next three days is to read diffs. An agent that applies edits without showing them to you removes the only control that protects the four invariants — and removes the only mechanism by which you learn the code well enough to defend it on camera.

GATE 0 — DO NOT PASTE MODULE 1 UNTIL

- ☐ Python ≥ 3.11 and SQLite ≥ 3.38 confirmed, `json_each` works
- ☐ Auto-accept is off; you will see every diff
- ☐ Folder is empty apart from `docs/`, `.cursorrules`, `.gitignore`
- ☐ No file from any previous attempt has been copied in
- ☐ `.gitignore` committed *before* anything else
- ☐ Virtual environment created and activated

Registry, Schema & the Query Ledger

The foundation. Invariant I3 becomes a primary key, and the push channel that makes the entire mid-flight mechanism possible gets built here.

PRE-FLIGHT

- ☐ Gate 0 complete; venv active
- ☐ docs/Master_Vibecoding_Plan_Alert_Routing_Agent.pdf present and openable by the agent
- ☐ .cursorrules at the repo root
- ☐ You have read the addendum below — **all seven items**
- ☐ No API key needed. Do not create .env yet.

ADDENDUM — READ BEFORE PASTING

Seven things that will otherwise cost you time. Items 1–4 are defects that were present in v1 of this plan and have been corrected in the prompt below; items 5–7 are traps in the tooling rather than in the design.

#	THE TRAP	WHY, AND WHAT TO DO
1	The foreign key with nothing to reference	evaluations.alert_id references alerts , and FKs are enforced — but nothing persists the alert, because AlertEvent is a Pydantic object in memory. The first ledger insert dies. An idempotent ensure_alert() must run before any pull.
2	The exception wrapper that lies	A blanket except IntegrityError → DuplicateQueryError also swallows the FK failure above, every CHECK violation, and Module 3's idempotency-key collision. Three unrelated bugs, one misleading message. Discriminate on exc.orig ; re-raise everything else.
3	The pragma that silently stops applying	PRAGMA foreign_keys is per-connection. Set in init_db() it covers one connection and no others. It must be a connect event listener — see the box in §3.1.
4	NOT NULL columns nothing can fill yet	qualification and ladder_rank are written at pull time, but scoring is Module 2. Insert sentinels (0.0 , -1); build_ladder() updates them. reachability is known now — it is a property of the observation, not of the ranking.
5	:memory: loses the schema	Each pooled connection gets its own empty database. init_db() creates tables on connection 1; the test queries connection 2 and gets no such table . Use a tmp_path file DB, or StaticPool with check_same_thread=False .
6	The async iterator that hangs pytest	A subscribe() generator that never terminates makes async for block forever at teardown — pytest hangs with no output at all, which reads like a crash. Give the bus a close() that pushes a sentinel to every subscriber.
7	Filtering the domain query by status	It looks like an obvious optimisation. It would drop Elena (offline L5) from the ladder and make Appendix A's floor and escalate scenarios unreachable — and you would not discover it until Module 4. The pull returns everyone whose domain matches, regardless of availability.

ONE DECISION WORTH UNDERSTANDING Push never touches evaluations — not an insert, not an update. Patched knowledge lives in DispatchState.observed and in CandidateSnapshot.source="push" ; the narrative of a push lives in audit_events . That is what makes the ledger a true count of questions asked, and it gives you a clean one-liner in the walkthrough: one row per person per alert, and every row says pull .

Build the foundation for the alert routing agent described in

`docs/Master_Vibecoding_Plan_Alert_Routing_Agent.pdf`. Create the repository layout from §2.3, with `pyproject.toml` and `requirements.txt` exactly as specified in §2.4 — including the `[tool.pytest.ini_options]` block and `pytest-asyncio>=0.24`.

1. **`alert_router/config.py`** — all tunables as module constants: `LATENCY_MS_RANGE = (120, 300)`, `RNG_SEED`, `DB_URL`. No numeric constant may live anywhere else, starting now.
2. **`alert_router/models_orm.py`** — SQLAlchemy 2.0 declarative models for all six tables in §3.1 exactly as specified. Write every `CHECK` as an explicit `CheckConstraint` in `__table_args__` — a plain `String` column silently drops it. The composite `PRIMARY KEY (alert_id, stakeholder_id)` on `evaluations` and the `UNIQUE` on `dispatch_attempts.idempotency_key` are load-bearing: they are invariants I3 and I2. Do not relax them for convenience.
3. **`alert_router/db.py`** — async engine over `aiosqlite`, session factory, and `init_db()`.
 - Register `PRAGMA foreign_keys=ON`, `PRAGMA journal_mode=WAL` and `PRAGMA busy_timeout=5000` in a `connect` event listener on the sync engine — **not** inside `init_db()`. The pragma is per-connection; setting it once leaves every later pooled connection unenforced, silently.
 - `init_db()` creates the schema, seeds stakeholders from `data/seed_stakeholders.json`, **and** seeds `channel_health` with one `healthy=1` row for every `(stakeholder, channel)` in that person's `channel_order`, so later "is a healthy fallback available?" queries are a plain `SELECT` rather than a missing-row special case.
4. **`alert_router/schemas.py`** — the Pydantic v2 models from §3.2, complete, including the `_must_actually_breach` validator, the `Channel.is_persistent` property, `AttemptRecord`, `AuditEvent` and `AttemptState`. JSON array columns convert to **tuples**, not lists — a list breaks frozen hashability.
5. **`alert_router/registry.py`**, with a clear pull/push split.

Pull (counted, latency-bearing):

- `async ensure_alert(alert)` — idempotent `INSERT ... ON CONFLICT DO NOTHING` into `alerts`. Call it at the top of every pull. Without it the first `evaluations` insert fails on a foreign key.
- `async query_by_domain(alert) -> list[CandidateSnapshot]` — ONE `SELECT` matching `primary_domain = :domain` OR a JSON-contained secondary domain via `json_each`, with a **bound parameter**, never an f-string. Apply the injectable simulated latency, then `INSERT` one `evaluations` row per matched person with `source='pull'`, real `observed_status / observed_at / reachability`, and sentinels `qualification=0.0`, `ladder_rank=-1` (Module 2 updates those). **The query must NOT filter on status** — offline people must appear in the result, or later scenarios become unreachable.
- `async fetch_one(alert, stakeholder_id)` — same accounting, single person.
- A second pull for an existing `(alert_id, stakeholder_id)` raises `DuplicateQueryError` — but **only** when `exc.orig` identifies the `evaluations` primary key. Foreign-key failures, `CHECK` failures and every other `IntegrityError` must propagate unchanged. Do not widen the except clause.
- Convert ORM rows to Pydantic and close the session **before returning**. A snapshot holding an attached ORM object would track the database, which destroys the entire point of a point-in-time observation.

Push (free, uncounted):

- `PresenceBus` on `asyncio.Queue`, with `publish(InterruptEvent)` and `subscribe() -> AsyncIterator[InterruptEvent]` supporting multiple independent subscribers — one queue per subscriber, fan-out via `put_nowait`.

- `subscribe()` must register the queue **synchronously at call time**, not on first iteration, or a listener can miss an event fired between task creation and first `__anext__`. Expose it as an async context manager so teardown unregisters in a `finally`.
- `close()` pushes a sentinel to every subscriber so `async for` loops terminate. Without this, tests hang with no output.
- `set_status()` and `set_channel_health()` write the database and then publish — **publishing happens outside any held lock**.
- Push writes **nothing** to `evaluations`, neither insert nor update.

6. **data/seed_stakeholders.json** — 10 stakeholders across `infrastructure`, `security`, `logistics`, `procurement`, `networking`. These four shapes are required, with these exact qualification totals under §3.4:

- Priya Raman — L3, primary infrastructure, on-call, online → **139**
- Tom Beckett — L1, primary infrastructure, on-call, online → **123**
- Elena Fischer — L5, primary infrastructure, not on-call, offline → **140**
- One L4, online, whose infrastructure match is *secondary* only → **87**

Additionally: **exactly 7 of the 10 must match infrastructure** (2 primary + 5 secondary), and **no other online person may reach qualification ≥ 120** , or the `floor` scenario silently degrades into a plain re-route. Provide an `async set_on_call(stakeholder_id, value)` helper so a scenario can drop Tom to 108 without a second seed file.

Use dependency injection for the clock and the latency generator — the registry takes `clock: Callable[[], float]` and `latency: Callable[[], float]`, defaulting to `time.time` and a `random.Random(RNG_SEED)`-backed generator — so tests can inject a zero-latency stub and stay fast and deterministic.

Comment **why**, not what — especially on every constraint and on any line that exists to preserve an invariant.

Then write `tests/conftest.py` and `tests/test_registry.py` and run them. Use a `tmp_path` file database or `StaticPool`; `plain :memory:` gives each pooled connection an empty database. Ten tests:

- `test_seed_loads_ten_stakeholders`
- `test_domain_query_matches_primary_and_secondary`
- `test_domain_query_includes_offline_people` — Elena must be in the result
- `test_infrastructure_matches_exactly_seven`
- `test_second_pull_same_alert_raises_duplicate_query_error`
- `test_different_alert_may_pull_same_person_once` — the budget is per alert, not global
- `test_foreign_key_violation_raises_integrity_not_duplicate` — an evaluation for a nonexistent alert must raise `IntegrityError`, **not** `DuplicateQueryError`
- `test_pragma_foreign_keys_on_for_every_connection` — assert on a second, freshly checked-out session
- `test_push_event_does_not_write_evaluations_row` — proves push is genuinely free
- `test_snapshot_unchanged_after_registry_mutation` — the cached observation must not track the database

READ THE DIFF — BEFORE YOU RUN ANYTHING

Seven checks, most of them one line. Do these **before** the tests: a green suite makes you stop looking.

```
# 1 – the pragma must be a CONNECTION listener, not a one-shot inside init_db()
grep -n "listens_for" alert_router/db.py
grep -n "foreign_keys" alert_router/db.py

# 2 – the exception wrapper must discriminate, not blanket-catch
grep -n "except IntegrityError" -A 6 alert_router/registry.py

# 3 – the domain query must NOT filter on availability
grep -n "status" alert_router/registry.py

# 4 – the alert row must be ensured before the first ledger insert
grep -n "ensure_alert" alert_router/registry.py

# 5 – push must never touch the query ledger
grep -n "evaluations" alert_router/registry.py

# 6 – the bus must terminate, and must not await under a lock
grep -n "put_nowait\|def close\|sentinel" alert_router/registry.py

# 7 – constraints must exist as real objects, not as comments
grep -n "CheckConstraint\|PrimaryKeyConstraint\|UniqueConstraint" alert_router/models_orm.py
```

#	ACCEPT IF	REJECT IF
1	A <code>@event.listens_for(engine.sync_engine, "connect")</code> handler issues all three pragmas	<code>PRAGMA foreign_keys</code> appears only inside <code>init_db()</code> — it will apply to one connection and no others
2	The handler reads <code>exc.orig</code> and re-raises anything that is not the evaluations primary key	A bare <code>except IntegrityError:</code> that always raises <code>DuplicateQueryError</code> — it will disguise FK and CHECK failures as duplicate queries
3	<code>status</code> appears only where the snapshot is built	<code>status</code> appears in the SELECT's <code>WHERE</code> — offline people vanish from the ladder and Modules 4–6 break silently
4	Called at the top of both <code>query_by_domain</code> and <code>fetch_one</code>	Missing, or called after the insert
5	Every hit is inside a pull path	Any hit inside <code>set_status</code> , <code>set_channel_health</code> or <code>publish</code> — push must be free
6	<code>put_nowait</code> for fan-out; a <code>close()</code> that sends a sentinel to every subscriber	<code>await queue.put(...)</code> inside a held lock, or no <code>close()</code> at all — the second one hangs pytest with no output
7	Every CHECK from §3.1 present, plus the composite PK and the UNIQUE	Plain <code>String</code> columns where a CHECK should be — the constraint is then documentation, not enforcement

Then read by eye, because grep cannot see these: the session is closed **before** snapshots are returned (no attached ORM object escapes into a frozen model); the evaluations insert writes sentinels `0.0` and `-1`; and the comments explain *why* a constraint exists rather than restating what the line does.

VERIFY — RUN IT YOURSELF, THEN COMMIT

1 · RUN THE GATE, THREE TIMES

```
pip install -r requirements.txt
pytest tests/test_registry.py -v
for i in 1 2 3; do pytest tests/test_registry.py -q || echo "FLAKE on run $i"; done
```

Expect **10 passed**. If it hangs with no output at all, it is the `PresenceBus` async iterator (addendum item 6), not a logic bug. Async ordering problems are intermittent by nature — a suite that passed once has not been tested.

2 • VERIFY THE DATABASE BY HAND

This is the step that proves the invariants are structural rather than documented. It is also, almost verbatim, your answer when a reviewer asks how you know you never query twice.

```
python -c "import asyncio; from alert_router.db import init_db; asyncio.run(init_db())"
sqlite3 alert_router.db
```

```
.schema evaluations      -- PRIMARY KEY (alert_id, stakeholder_id) must be in the DDL
.schema dispatch_attempts -- idempotency_key ... UNIQUE must be in the DDL
.schema stakeholders     -- every CHECK must be present
PRAGMA foreign_keys;      -- 1
SELECT count(*) FROM stakeholders; -- 10
SELECT count(*) FROM channel_health; -- > 0, seeded
SELECT count(*) FROM stakeholders WHERE primary_domain='infrastructure'
OR EXISTS (SELECT 1 FROM json_each(secondary_domains) WHERE value='infrastructure'); -- 7
.quit
```

IF A CHECK IS MISSING `create_all()` dropped it and the invariant is fiction. Fix it here, not in Module 2. The same applies if `PRAGMA foreign_keys` returns 0 — the listener is not registered.

3 • RESET HYGIENE

```
rm -f alert_router.db*      # the * matters: WAL leaves -wal and -shm sidecar files
```

There is no Alembic. `init_db()` seeds on first run only, so a stale database **will not pick up seed changes** — this is the single most likely source of baffling failures in Modules 2–6.

4 • COMMIT

```
git add -A
git commit -m "feat(module-1): SQLite registry, query ledger, PresenceBus – Gate 1 green (10/10)"
```

EXPLAIN IT BACK — OUT LOUD, WITHOUT LOOKING

1 • Why does the registry have two verbs?

"The brief says detect a change mid-flight, and also says do not re-query anyone you have already evaluated. You cannot notice a change you have forbidden yourself to look for. So a pull is a question — counted, one per person per alert — and a push is an announcement that carries the new state in its payload. Knowledge refreshes; the query counter does not move."

2 • How do you know you never query twice?

"It is the primary key of the `evaluations` table. A second pull raises `IntegrityError`. It is not a convention I follow, it is a write the database refuses."

3 • Why must the snapshot not track the database?

"Because the divergence between what the agent believes and what is true is the mid-flight signal. If the snapshot silently refreshed, there would be no change left to detect and nothing for a push event to tell us."

If any of those three needs the document open in front of you, re-read the addendum and the diff before starting Module 2. This is the module everything else rests on, and it is the one the reviewer will probe hardest.

GATE 1

- ☐ All seven diff checks pass
- ☐ You can give the three answers above unaided
- ☐ 10/10 green on three consecutive runs
- ☐ `.schema` shows every CHECK, the composite PK, and the UNIQUE
- ☐ `PRAGMA foreign_keys` returns 1 on a fresh connection
- ☐ Infrastructure returns exactly 7 candidates, offline people included
- ☐ No numeric constant lives outside `config.py`
- ☐ Nothing in the diff removed or relaxed a constraint
- ☐ `git status` clean; no `.db` or `.env` staged

STATE THIS OUT LOUD BEFORE CONTINUING Why is `test_snapshot_unchanged_after_registry_mutation` passing the precondition for the entire mid-flight mechanism? — *Because the snapshot is a frozen point-in-time observation that does not track the database, a divergence can exist between what the agent believes and what is true; that divergence is the mid-flight signal itself, and if the snapshot silently refreshed there would be no change left to detect and nothing for a push event to tell us.* Put this sentence in the README.

Scoring, the Ladder & DispatchState

Where invariant I4 becomes arithmetic. The ladder is frozen here and never grows again — that is what makes I3 structural rather than aspirational.

PRE-FLIGHT

- ☐ Gate 1 green, committed
- ☐ `rm -f alert_router.db*` — you are about to change how evaluations are written
- ☐ venv active; no API key needed
- ☐ Addendum below read — **item 2 in particular**, because it is a test that can pass while proving the opposite of what it claims

ADDENDUM — READ BEFORE PASTING

#	THE TRAP	WHY, AND WHAT TO DO
1	Ranking that needs a database	If <code>build_ladder()</code> writes <code>ladder_rank</code> itself, every ranking test needs an engine and the scoring logic becomes untestable in isolation. Keep <code>ranking.py</code> pure — snapshots in, <code>DispatchPlan</code> out — and put the write-back in a separate <code>persist_ladder(session, plan)</code> .
2	The I4 test that proves the opposite	Sorting is <code>(reachability, qualification, seniority_tier)</code> , so the online junior legitimately sorts <i>first</i> . A test that asserts "the director outranks the junior" against sort order will fail, and the tempting fix is to reorder the sort key — which destroys I4. The test must assert on <code>qualification</code> directly (<code>140 > 123</code>), and assert the sort order separately , with a comment saying the junior leads only because of reachability. Two assertions, two different claims.
3	<code>next_candidate()</code> skipping the wrong ledger	It must skip everyone in <code>attempted</code> — including aborted attempts — not just notified . An aborted person's idempotency key is already taken, so re-offering them surfaces as an <code>IntegrityError</code> at the worst possible moment, mid-reroute.
4	Re-scoring that quietly re-resolves	<code>rescore()</code> recomputes <code>ScoreBreakdown</code> values from <i>patched</i> observations. It must not add members, and it must not renumber <code>rank</code> . Reacting to someone becoming available is the job of the derived <code>BETTER_MATCH</code> event in Module 3, not of silently reshuffling a frozen ladder that the executor is walking. Re-scoring is legal because it uses facts already paid for; re-resolving would need a new pull.
5	Non-monotonic audit sequence	In Module 3 the executor and the interrupt listener both write audit rows. If <code>seq</code> is allocated outside the lock, two writers collide on <code>UNIQUE (alert_id, seq)</code> . Allocate it inside <code>async with self.lock</code> , now, while there is only one writer and the bug is invisible.
6	<code>patch_from_push</code> on a stranger	Cross-domain presence traffic names people who were never evaluated. Indexing <code>self.observed[id]</code> blindly raises <code>KeyError</code> on entirely normal events. Return <code>False</code> instead — that is matrix row R1, arriving early.
7	A floor message with no numbers in it	<code>clears_floor()</code> returns <code>(bool, str)</code> and that string goes straight into the audit trail, the notification, and the video. " <i>candidate not qualified</i> " is worth nothing. " <i>qualification 108 < floor 120 (max of CRITICAL minimum 120, incumbent 139 – tolerance 25)</i> " is the whole argument, visible.

THE PROMPT — PASTE THIS INTO CURSOR

Build the ranking layer and the state container on top of Module 1's registry, per docs/Master_Vibecoding_Plan_Alert_Routing_Agent.pdf §3.3 and §3.4.

1. **Extend `alert_router/config.py`** with `DOMAIN_POINTS`, `SENIORITY_POINTS`, `ON_CALL_POINTS`, `MIN_QUALIFICATION`, `DOWNGRADE_TOLERANCE`. Nothing numeric may be hardcoded elsewhere — a reviewer will want to change a weight and re-run.
2. **`alert_router/ranking.py` — pure, no database access.**
 - `score(snapshot, alert) -> ScoreBreakdown` implementing §3.4 exactly. **Critical: qualification must contain no availability term.** Availability appears only as the separate `reachability` field. If you find yourself adding points for being online, stop — that is invariant I4 breaking. A `domain_fit` of "none" means ineligible, not zero points.
 - `build_ladder(snapshots, alert) -> DispatchPlan` — filter ineligible, sort by (`reachability`, `qualification`, `seniority_tier`) descending, assign ranks, freeze into a tuple.
 - `clears_floor(candidate, incumbent, severity) -> tuple[bool, str]` — the verdict plus a human-readable reason containing **actual numbers**, e.g. "qualification 108 < floor 120 (max of CRITICAL minimum 120, incumbent 139 – tolerance 25)". That string goes into the audit trail and the video, so make it good.
3. **`alert_router/state.py` — DispatchState per §3.3:**
 - The four ledgers (`evaluated`, `observed`, `attempted`, `notified`) plus `suppressed`, and the per-alert `asyncio.Lock`.
 - `patch_from_push(ev) -> bool` — returns `False` for a subject not in `observed` rather than raising.
 - `record_audit(kind, actor, summary, payload)` — append-only, allocating a monotonic seq **inside the lock**, and writing the `audit_events` row.
 - `next_candidate()` — walks the frozen ladder in rank order, skipping anyone in `attempted` (**including aborted**) or `suppressed`.
 - `mark_suppressed(id, why)` — `why` must be the numeric string from `clears_floor`.
 - `rescore()` — recomputes scores from *patched* observations without touching ladder membership or rank numbering. Comment clearly on why re-scoring is legal but re-resolving is not.
4. **`persist_ladder(session, plan)`** in `ranking.py` or `state.py` — UPDATES the `qualification` and `ladder_rank` sentinels written in Module 1. This is an UPDATE to the ledger's derived columns, not a new evaluation: it must not insert rows and must not change `source`.

Then write and run `tests/test_ranking.py`:

- `test_primary_domain_l3_outranks_secondary_domain_l4` — domain fit dominates seniority
- `test_offline_director_beats_online_junior_on_qualification` — **this is I4**. Assert on `qualification` directly (140 > 123). Then, in a *separate* assertion, confirm the online junior sorts first and comment that this is because of `reachability` alone, not because he is more qualified.
- `test_qualification_ignores_availability` — score the same person twice, once online and once offline; `qualification` must be identical and only `reachability` may differ
- `test_clears_floor_rejects_underqualified_with_numeric_reason` — Tom at 108 against incumbent 139 on CRITICAL: rejected, and the reason string contains both 108 and 120
- `test_clears_floor_accepts_when_within_tolerance` — Tom at 123 accepted
- `test_next_candidate_skips_aborted_attempts`
- `test_rescore_does_not_change_ladder_membership_or_rank`
- `test_patch_from_push_returns_false_for_unevaluated_subject`

- `test_audit_seq_is_monotonic_under_concurrent_writers` — spawn several tasks calling `record_audit` concurrently and assert the sequence has no gaps or duplicates

READ THE DIFF — BEFORE YOU RUN ANYTHING

```
# 1 – ranking must be PURE: no session, no await, no database
grep -n "session\\|select(\\|AsyncSession\\|await " alert_router/ranking.py

# 2 – availability must not appear in the qualification sum
grep -n "reachability\\|status\\|online" alert_router/ranking.py

# 3 – the ladder walk must skip ATTEMPTED, not merely NOTIFIED
grep -n "attempted\\|notified" alert_router/state.py

# 4 – the audit sequence must be allocated inside the lock
grep -n "_seq" -B 3 -A 3 alert_router/state.py

# 5 – the floor reason must carry numbers
grep -n "def clears_floor" -A 20 alert_router/ranking.py
```

#	ACCEPT IF	REJECT IF
1	No hits at all — <code>ranking.py</code> takes snapshots and returns a plan	Any session or <code>await</code> . Scoring then cannot be tested without a database, and the layer split has collapsed
2	<code>reachability</code> is assigned to its own field and used only in the sort key	Availability appearing anywhere in the <code>qualification</code> arithmetic — this is invariant I4 breaking , and it is the exact bug the brief warns about
3	<code>next_candidate()</code> tests membership of <code>attempted</code> and <code>suppressed</code>	It tests only <code>notified</code> — an aborted person is then re-offered, and their idempotency key is already taken
4	The increment sits inside <code>async</code> with <code>self.lock</code>	Allocated outside — two writers collide on <code>UNIQUE (alert_id, seq)</code> as soon as Module 3 adds a second writer
5	The string interpolates the candidate's score, the floor, and where the floor came from	A static message like <code>"not qualified"</code> . This string goes on screen in the video, and it is the whole argument for your design

Read by eye: `rescore()` must not renumber `rank` or add members, and it should carry a comment explaining why re-scoring is legal — it uses facts already paid for — while re-resolving is not, because that would need a new pull.

VERIFY — RUN IT YOURSELF, THEN COMMIT

```
pytest tests/ -v # Module 1 tests must still pass
for i in 1 2 3; do pytest tests/ -q || echo "FLAKE $i"; done
```

Then check the write-back landed, since sentinels that never get updated are silent:

```
sqlite3 alert_router.db "SELECT stakeholder_id, qualification, ladder_rank
FROM evaluations ORDER BY ladder_rank;"
```

No row may still show `ladder_rank = -1` for an eligible candidate, and Priya must read 139, Tom 123, Elena 140.

```
git commit -am "feat(module-2): scoring, frozen ladder, DispatchState – Gate 2 green"
```


1 · Why does qualification contain no availability term?

"Because if presence were an addend, anyone going offline would mechanically demote themselves and the next person up would be whoever happens to be at their desk. That is exactly the failure the brief calls out. Availability lives on a separate axis called reachability, so being online cannot raise anyone's score."

2 · Then why does the online junior sort above the offline director?

"Because the sort key is reachability first — an unreachable person cannot be the initial recipient, so there is no point ranking them first. But every replacement comparison uses qualification alone, and the director is still the more qualified person. Ordering for first contact and ranking for competence are different questions."

3 · Why is re-scoring allowed but re-resolving forbidden?

"Re-scoring recomputes from observations I already paid for, so it costs no queries. Re-resolving would add someone who was never evaluated, and learning anything about them requires a pull — which the budget forbids. That is why the ladder's membership is frozen while its scores can move."

GATE 2

- ☐ All five diff checks pass
- ☐ You can give the three answers above unaided
- ☐ All tests green, three consecutive runs
- ☐ `qualification` is provably independent of availability — the dedicated test exists
- ☐ Sort-order and qualification claims are asserted *separately*
- ☐ The floor rejection string contains real numbers
- ☐ `ladder_rank` sentinels have been updated in the database
- ☐ Nothing in `ranking.py` touches a session

The Executor, the Commit Point & Interrupts

The hardest module, and the one the assessment is really about. Budget the most time here. Everything before this was scaffolding for the abort window.

PRE-FLIGHT

- ☐ Gates 1 and 2 green, both committed
- ☐ `rm -f alert_router.db*`
- ☐ You can state, without looking, what happens to an in-flight dispatch when the recipient goes offline on SMS versus on Slack (rows R2 and R3)
- ☐ Addendum below read in full. **Items 1 and 2 are the difference between a working abort and a hung process.**

ADDENDUM — READ BEFORE PASTING

#	THE TRAP	WHY, AND WHAT TO DO
1	Shielding too much	<code>asyncio.shield()</code> must wrap only the commit write . Wrapping the send makes the dispatch uncancellable and the abort window disappears — the demo then has nothing to show. The shield exists to mark exactly where the one-way door closes: before it, cancellation is clean; after it, the row is written even if the task is being torn down.
2	Swallowing <code>CancelledError</code>	Catching it to write the <code>aborted</code> row is correct. Not re-raising it is a bug — inside a <code>TaskGroup</code> a swallowed cancellation breaks shutdown and can hang the process. Distinguish deliberate aborts from real cancellation with an explicit flag on the attempt: catch, write <code>aborted</code> , and re-raise unless we requested it.
3	Reserving after you start sending	The <code>dispatch_attempts</code> row must be inserted and committed before the first byte moves. That ordering is what makes an aborted attempt block a later re-attempt — the idempotency key is taken from reservation, not from delivery. Reserve late and I2 has a window.
4	Sharing an <code>AsyncSession</code> across tasks	It is not concurrency-safe. The executor task and the listener task each need their own session from the factory. Symptoms are surreal — rows appearing twice, transactions interleaving, <code>InvalidRequestError</code> at random. WAL and <code>busy_timeout</code> from Module 1 handle contention; they do not make a session shareable.
5	Timing-based tests	"Inject the interrupt at <code>t+0.8 s</code> " is fine for the demo and fatal for the suite. Tests must drive phase transitions with an <code>asyncio.Event</code> the executor awaits at a known point, so the interrupt lands <i>deterministically</i> pre-commit or post-commit. A test that depends on <code>sleep</code> ordering will pass on your laptop and fail once in twenty runs.
6	Re-routing to the person you just aborted	<code>next_candidate()</code> already skips <code>attempted</code> , but the executor must add the aborted person to that ledger <i>before</i> asking for the next one. Off-by-one here produces an <code>IntegrityError</code> on the idempotency key, which reads as a duplicate bug when it is really a ladder-walk bug.
7	A failed connect recorded as an abort	<code>failed</code> and <code>aborted</code> are different outcomes with different stories: the transport refused us, versus we changed our mind. Both are terminal and both keep the person in <code>attempted</code> , but conflating them makes the audit trail lie.

THE SENTENCE TO BE ABLE TO SAY *"Dispatch is reserved, connecting, in-flight, committed. Everything before the commit is abortable and the abort is clean, because nothing has left the building. The shield wraps only the commit write, so once that row lands the message exists and the only honest response to new information is a supplement, never a retraction."*

Build the dispatch executor, the channel adapters and the interrupt listener, per docs/Master_Vibecoding_Plan_Alert_Routing_Agent.pdf §1.3 and §3.6.

1. **alert_router/channels.py** — an adapter per channel with `connect()` and `send()`, both async and both with injectable latency and injectable failure, so scenarios are deterministic. Classification comes from `Channel.is_persistent`: Slack is synchronous, email and SMS are persistent. Expose `healthy_channels(session, stakeholder)` reading `channel_health` and returning them in `channel_order`.
2. **alert_router/executor.py** — the two-phase send:
 - **RESERVE**: insert `dispatch_attempts` with `state='reserved'` and the idempotency key `{alert_id}:{stakeholder_id}`, and **commit it before sending**. A collision here means this person was already attempted — surface it as a distinct, named exception, not as a generic integrity failure.
 - **CONNECTING** → **IN_FLIGHT** → **COMMITTED**, each transition written and each legal transition per §3.6 enforced. A refused connect is `failed`, never `aborted`.
 - **The commit point**: `asyncio.shield()` wraps **only** the write that sets `state='committed'` and `committed_at`. Nothing else is shielded.
 - Cancellation: catch `CancelledError`, write `state='aborted'` with an `outcome_reason`, add the person to `attempted`, and **re-raise unless the abort was deliberate** — use an explicit flag on the attempt to tell the two apart. Never swallow a cancellation you did not request.
 - Expose a test hook — an awaitable barrier the executor waits on at a known phase — so tests can land an interrupt deterministically pre-commit or post-commit without relying on sleep timing.
 - Each concurrent task gets its **own** session from the factory. Never share an `AsyncSession` between tasks.
3. **alert_router/interrupts.py**:
 - A listener that subscribes to the `PresenceBus` **before** dispatch begins and applies each event via `state.patch_from_push()`.
 - A relevance filter: events about people not in the ladder and not the incumbent are dropped as row R1 without waking the matrix.
 - Derivation of **BETTER_MATCH**: when a patched observation makes a ladder member reachable whose `qualification` exceeds the incumbent's, emit a derived **BETTER_MATCH** interrupt. This is derived from facts already paid for — it must never trigger a pull, and it may only name people already in the ladder.
 - Every applied event writes an **INTERRUPT** audit row with what changed and how we learned it.
4. **alert_router/agent.py** — top-level orchestration with `asyncio.TaskGroup`: one task running the dispatch, one running the listener, with clean teardown of both and of the bus subscription. Use `except*` for exception groups.

Then write and run `tests/test_executor.py`:

- `test_happy_path_reaches_committed`
- `test_reserve_row_exists_before_send_starts`
- `test_abort_pre_commit_writes_aborted_and_sends_nothing`
- `test_abort_after_commit_is_impossible` — the shielded write completes even when the task is cancelled during commit
- `test_aborted_attempt_blocks_reattempt_of_same_person` — I2 through the idempotency key
- `test_failed_connect_is_failed_not_aborted`
- `test_cancelled_error_is_reraised_when_not_deliberate`

- `test_listener_drops_events_for_unevaluated_people` — row R1
- `test_better_match_derived_without_any_new_pull` — assert the `evaluations` row count is unchanged
- `test_two_concurrent_tasks_do_not_share_a_session`

Run the suite **twenty times** and fix any flake before proceeding. A race condition that is not reliably reproduced is not handled — it is merely unobserved.

READ THE DIFF — BEFORE YOU RUN ANYTHING

This is the module where a plausible-looking diff can be quietly wrong, so read it properly. Four greps and one careful read of the send path.

```
# 1 – the shield must appear EXACTLY once, and only around the commit write
grep -n "shield" alert_router/executor.py

# 2 – CancelledError must be re-raised unless the abort was deliberate
grep -n "CancelledError" -A 10 alert_router/executor.py

# 3 – the reserve row must be committed before the send begins
grep -n "reserved\|commit()" -n alert_router/executor.py | head -30

# 4 – no session may be shared across tasks
grep -n "session" alert_router/executor.py alert_router/interrupts.py | head -30

# 5 – a derived BETTER_MATCH must never trigger a pull
grep -n "query_by_domain\|fetch_one" alert_router/interrupts.py
```

#	ACCEPT IF	REJECT IF
1	One occurrence, wrapping the write that sets <code>state='committed'</code>	It wraps the send, the connect, or the whole attempt. The dispatch becomes uncancelable, the abort window disappears, and the demo has nothing to show
2	Caught, an <code>aborted</code> row written, then <code>raise</code> — guarded by an explicit deliberate-abort flag	Caught and swallowed. Inside a <code>TaskGroup</code> that breaks shutdown and hangs the process at teardown
3	The insert and its commit both precede the first adapter call	Reserved after sending starts — I2 then has a window in which two attempts can both believe they are first
4	Each task takes its own session from the factory	One session captured and used by both the executor and the listener. Symptoms are surreal: duplicated rows, interleaved transactions, random <code>InvalidRequestError</code>
5	No hits — derivation reads only <code>state.observed</code>	Any pull inside the listener. That is invariant I3 broken, and it is the single claim the brief cares most about

Then read the send path top to bottom and satisfy yourself you can name, for each line, which state the attempt is in. If you cannot narrate that path, ask for it to be commented until you can — you are going to walk a reviewer through it on camera.

VERIFY — RUN IT YOURSELF, THEN COMMIT

```
for i in $(seq 1 20); do pytest tests/test_executor.py -q || echo "FLAKE on run $i"; done
```

Twenty clean runs, no exceptions. If the suite ever hangs, it is a subscription that was never closed or a `CancelledError` that was swallowed (addendum items 2 and 6) — not a slow test.

```
sqlite3 alert_router.db "SELECT stakeholder_id, state, role, outcome_reason
                        FROM dispatch_attempts ORDER BY id;"
```

Every row should tell a coherent story: one `aborted` with a reason, one `committed`. Two committed rows for the same person is I2 broken.

```
git commit -am "feat(module-3): executor, commit point, interrupt listener — Gate 3 green (20x)"
```

EXPLAIN IT BACK — OUT LOUD, WITHOUT LOOKING

1 • Where is the commit point, and why there?

"Dispatch goes reserved, connecting, in-flight, committed. Everything before the commit is abortable and the abort is clean, because nothing has left the building. The shield wraps only the commit write, so once that row lands the message exists — and the only honest response to new information after that is a supplement, never a retraction."

2 • Why does an aborted attempt still block a re-attempt?

"Because the idempotency key is taken at reservation, not at delivery. The row exists from the moment we commit to trying, so the database will refuse a second attempt on the same person for the same alert whether the first one succeeded or was cancelled."

3 • How does the listener know someone became a better match without asking?

"It does not ask. The presence event carries the new state, we patch the cached snapshot from the payload, and we re-score against observations already paid for. If that makes a ladder member outrank the incumbent, we derive a `BETTER_MATCH` interrupt. The evaluations table is untouched — you can watch the query count stay flat while the decision changes."

GATE 3

- ☐ All five diff checks pass
- ☐ You can give the three answers above unaided, and narrate the send path
- ☐ Twenty consecutive green runs of `test_executor.py`
- ☐ `asyncio.shield()` appears exactly once, around the commit write only
- ☐ An aborted attempt provably blocks a re-attempt of the same person
- ☐ A derived `BETTER_MATCH` adds zero rows to `evaluations`
- ☐ No `CancelledError` is swallowed anywhere
- ☐ No `AsyncSession` is shared between tasks

The Decision Matrix

Eleven ordered rows, first match wins. Mechanical if Module 3 is correct — and the single most reviewable artefact in the repository.

PRE-FLIGHT

- ☐ Gate 3 green, twenty clean runs, committed
- ☐ `rm -f alert_router.db*`
- ☐ §3.5 open in front of you — the prompt refers to rows by number, not by description
- ☐ Addendum read; **item 1 is the one that silently breaks the table**

ADDENDUM — READ BEFORE PASTING

#	THE TRAP	WHY, AND WHAT TO DO
1	Nested <code>if</code> blocks instead of an ordered list	Nesting is how R2 gets shadowed by R3 with nobody noticing — and the failure is silent, because the wrong row still returns a plausible action. Implement the matrix as a literal ordered sequence of <code>(predicate, handler, row_id)</code> tuples, iterated top to bottom. The table in §3.5 and the code should be readable side by side.
2	R2 and R5 differing only by phase	Both fire on <i>presence</i> → <i>offline</i> . R2 is pre-commit , R5 is post-commit . If the phase predicate is missing or loose, a post-commit drop on SMS matches R2 and the parallel escalation never happens — the alert quietly under-delivers. Test the same event in both phases explicitly.
3	A silent default when nothing matches	Falling through to <code>CONTINUE_UNCHANGED</code> makes every future gap in the table invisible. R11 is a real row with a real condition; a genuine fall-through is a bug and should raise.
4	R9 naming someone outside the ladder	The escalation target must already be a ladder member with a paid-for observation. Naming anyone else means a new pull, which breaks I3. Assert ladder membership inside the handler, not just in the caller.
5	R4 doing only half its job	R4 is invariant I4 and has three obligations, all required: keep the incumbent but move them to a <i>persistent</i> channel; page the highest-qualification ladder member; and record the rejected junior in <code>suppressed</code> with the numeric reason from <code>clears_floor</code> . Implementations routinely do the first two and skip the third — and the third is the one that appears on screen in the video.
6	Untagged decisions	Every <code>RoutingDecision</code> carries <code>matrix_row</code> , including the boring <code>CONTINUE_UNCHANGED</code> ones. Without it the audit trail cannot be traced back to the table, and the video loses its best move: pointing at a line and saying "that row fired".

THE PROMPT — PASTE THIS INTO CURSOR

Implement `alert_router/decisions.py` — the decision matrix from `docs/Master_Vibecoding_Plan_Alert_Routing_Agent.pdf` §3.5, rows R1 through R11.

1. Structure it as a **literal ordered sequence** of `(row_id, predicate, handler)` entries, evaluated top to bottom with first-match-wins semantics. Do not use nested `if` blocks — the ordering is the specification, and nesting hides shadowed rows.

2. `decide(state, event, attempt) -> RoutingDecision`. Every returned decision carries `matrix_row` `("R1" ... "R11")` and a one-sentence `rationale` written for a human reading an audit trail. If no row matches, **raise**

— do not default to `CONTINUE_UNCHANGED` .

3. Phase is a first-class predicate input. R2 and R5 differ only by pre-commit versus post-commit; derive the phase from the attempt's `AttemptState` , and make it impossible for a post-commit event to match a pre-commit row.
4. **R4 has three obligations**, all of them required: keep the incumbent but switch them to a persistent channel; page the highest-qualification ladder member as `escalate_to_id` ; and populate `suppressed` with `(stakeholder_id, reason)` where the reason is the numeric string from `clears_floor()` .
5. R6 must reuse the **same idempotency key** — same person, different pipe, one notification. R9's escalation target must already be a ladder member; assert that inside the handler.
6. R11 is terminal and loud; mark the state terminal, write an audit row, and set the alert's `state` to `exhausted` . A silent drop is the worst possible outcome for an alerting system.
7. Wire the matrix into `agent.py` so an interrupt flows: `event` → relevance filter → `patch_from_push` → `decide()` → handler → audit row.

Then write and run `tests/test_decisions.py` . Every row gets at least one test, parametrised, and the parametrisation must fail loudly if a row has no case. Include specifically:

- `test_every_matrix_row_has_a_test` — collect the row ids from the matrix and assert the test set covers all eleven
- `test_r2_presence_drop_on_sms_is_a_non_event`
- `test_r3_presence_drop_on_slack_reroutes_when_floor_cleared`
- `test_r4_refuses_downgrade_and_escalates_up` — assert all three obligations, including the numeric suppression reason
- `test_r5_same_event_post_commit_escalates_in_parallel` — same event as R2, opposite phase, different row
- `test_r6_failover_keeps_same_idempotency_key`
- `test_r8_never_renotifies_someone_already_notified`
- `test_r9_escalation_target_is_always_a_ladder_member`
- `test_r10_low_severity_does_not_wake_a_director`
- `test_r11_exhaustion_is_terminal_and_recorded`
- `test_unmatched_event_raises_rather_than_defaulting`

READ THE DIFF — BEFORE YOU RUN ANYTHING

```
# 1 – the matrix must be an ORDERED SEQUENCE, not nested ifs
grep -n "R1\\|R2\\|R3\\|R11" alert_router/decisions.py | head -20

# 2 – there must be no silent fall-through
grep -n "return\\|raise" alert_router/decisions.py | tail -20

# 3 – phase must be a real predicate, separating R2 from R5
grep -n "pre_commit\\|post_commit\\|is_pre_commit\\|phase" alert_router/decisions.py

# 4 – R4 must do all three things
grep -n "R4" -A 25 alert_router/decisions.py
```

#	ACCEPT IF	REJECT IF
1	A literal list or tuple of <code>(row_id, predicate, handler)</code> , iterated top to bottom	Nested <code>if/elif</code> blocks. Ordering is the specification, and nesting is how a row gets shadowed with nobody noticing — the wrong row still returns a plausible action

2	An unmatched event <code>raise</code> s	A trailing <code>return CONTINUE_UNCHANGED</code> . Every future gap in the table then becomes invisible
3	R2 tests pre-commit and R5 tests post-commit explicitly	No phase test. A post-commit drop on SMS then matches R2 and the parallel escalation never happens — the alert quietly under-delivers
4	Incumbent moved to a persistent channel, a higher-qualification member set as <code>escalate_to_id</code> , and the rejected junior written into <code>suppressed</code> with the numeric reason	Only two of the three. The suppression line is the one that appears on screen, and it is routinely the one that gets skipped

Read by eye: every returned `RoutingDecision` carries a `matrix_row` , including the boring `CONTINUE_UNCHANGED` ones — without it the audit trail cannot be traced back to the table, and the video loses its best move.

VERIFY — RUN IT YOURSELF, THEN COMMIT

```
pytest tests/ -v
sqlite3 alert_router.db "SELECT seq, kind, actor, summary FROM audit_events ORDER BY seq;"
```

Read the audit trail as prose. It should narrate the incident well enough that someone who has never seen the code can follow it — that is the standard, because it is what goes on screen in the video.

```
git commit -am "feat(module-4): decision matrix R1-R11 – Gate 4 green"
```

EXPLAIN IT BACK — OUT LOUD, WITHOUT LOOKING

1 • Why is a presence drop during an SMS a non-event?

"Because SMS is persistent — it waits on the device. "Offline" means away from keyboard, not unreachable. Slack is synchronous, so the same event on Slack is a real problem. Treating those as identical causes a whole class of pointless re-routes, which is why channel persistence is a first-class property rather than a detail."

2 • Why does R4 refuse to re-route to the only person who is online?

"Because he is under-qualified. The floor is the higher of the severity minimum and the incumbent's score minus a tolerance — 120 here — and he scores 108. Routing to him would be escalating downward just because he happens to be available, which is the failure the brief names explicitly. So we keep the incumbent on a persistent channel, page someone more senior, and record the junior as suppressed with the arithmetic visible."

3 • Why does a channel failover not count as a duplicate?

"The idempotency key is alert plus stakeholder, and it deliberately excludes the channel. The same notification down a different pipe is one notification, so it collides with itself and is handled as a continuation rather than a second send."

GATE 4

- ☐ All four diff checks pass
- ☐ You can give the three answers above unaided
- ☐ All eleven rows have a passing test; the coverage test proves it
- ☐ R2 and R5 are distinguished by phase, with a test for each
- ☐ R4 satisfies all three obligations including the numeric suppression
- ☐ An unmatched event raises rather than defaulting
- ☐ The audit trail reads as a coherent narrative

Context Synthesis & the Explanation Layer

Invariant I1, made visible. The smallest module — and the first one where an API key does anything at all.

PRE-FLIGHT

- ☐ Gate 4 green, committed
- ☐ Optional: `cp .env.example .env` and add your `OPENAI_API_KEY`. **Confirm `.env` is in `.gitignore` first** — `git check-ignore -v .env` should print the matching rule
- ☐ The demo must still run with no key at all. That is a requirement, not a convenience

ADDENDUM — READ BEFORE PASTING

#	THE TRAP	WHY, AND WHAT TO DO
1	Letting the model decide anything	The LLM receives an envelope whose recipient, channel and role are already determined . It writes prose. It is given no tools, no candidate list to choose from, and no authority. If the routing decision can vary with model output, the system is non-deterministic and untestable — and the brief's requirement to defend every decision becomes impossible to meet.
2	Tests that hit the network	A suite that needs an API key is a suite the reviewer cannot run. Mock the client entirely. Include one test with the key absent and one with it mocked-present, and assert both produce a valid envelope with the same structured fields.
3	A fallback that hides itself	When the API errors or times out, fall back to the deterministic template — and record that you did , in the audit trail. Silent degradation is how you end up unable to explain your own output on camera.
4	Copying the alert instead of carrying it	I1 says the final envelope carries the <i>original AlertEvent</i> . The test should assert object identity or full equality after two re-routes, not that the fields look similar. Reconstructing an equivalent alert somewhere in the chain is exactly the bug this invariant exists to catch.
5	Vague <code>considered_and_passed</code> entries	" <i>Not selected</i> " is worthless. " <i>Tom Beckett — qualification 108 < floor 120</i> " is the entire argument for your design, in one line, in front of the reviewer.

On prompt injection: metric names and reasons flow into the model's context, so in principle a hostile metric name could try to steer the output. The blast radius is nil by construction — the model has no tools, no routing authority, and its output is prose displayed to a human. Worth one line in the README's defence notes, because noticing it is the point.

THE PROMPT — PASTE THIS INTO CURSOR

Implement `alert_router/context.py` — the context compiler and the explanation renderer, per `docs/Master_Vibecoding_Plan_Alert_Routing_Agent.pdf §3.2 (NotificationEnvelope)`.

1. `compile_envelope(state, attempt) -> NotificationEnvelope`, carrying:
 - the **original** `AlertEvent` object — carried by reference through every re-route, never rebuilt;
 - the full `audit_trail` as a tuple of human-readable summaries;
 - `chosen_because` — a numeric comparison against the runner-up, e.g. "*primary domain 100 + L3 seniority 24 + on-call 15 = 139, against Marcus Webb at 87 (secondary domain)*";
 - `considered_and_passed` — every suppressed or skipped person with a **numeric** reason.

2. `render(envelope) -> str` with two paths behind one interface:
 - **Deterministic template** — always available, no key, no network. This is the default and the one the tests assert on.
 - **OpenAI renderer** — used only when `OPENAI_API_KEY` is present. `temperature=0`, a capped `max_tokens`, and a short timeout. On any error or timeout, fall back to the template **and write an audit row recording that the fallback fired**.
 - The model is given the decided envelope and asked for prose. It is given no tools, no candidate list, and no authority to change the recipient, the channel or the role.
3. Wire it into `executor.py` so the committed dispatch produces a rendered envelope, and the CLI in Module 6 can display it.

Then write and run `tests/test_context.py`:

- `test_context_survives_two_reroutes` — **this is I1**. Priya → aborted → Tom → aborted → Elena; assert the envelope's alert equals the original exactly and the audit trail contains all three legs
- `test_envelope_names_runner_up_with_numbers`
- `test_suppressed_people_appear_with_numeric_reasons`
- `test_render_without_api_key_uses_template` — monkeypatch the key away
- `test_render_with_mocked_client_produces_same_structured_fields` — no network
- `test_api_failure_falls_back_and_records_it`

No test may make a network call.

READ THE DIFF — BEFORE YOU RUN ANYTHING

```
# 1 – the model must receive a DECIDED envelope, never a choice
grep -n "def render" -A 30 alert_router/context.py

# 2 – no test may reach the network
grep -rn "openai\|AsyncOpenAI\|http" tests/

# 3 – the original alert must be carried, not rebuilt
grep -n "AlertEvent(" alert_router/context.py alert_router/executor.py

# 4 – the fallback must announce itself
grep -n "except\|fallback\|record_audit" -A 3 alert_router/context.py
```

#	ACCEPT IF	REJECT IF
1	The prompt contains the already-chosen recipient, channel and role, and asks only for prose	A candidate list, a "pick the best" instruction, or any tool definition. The routing decision would then vary with model output — untestable, and impossible to defend
2	Only mocks and monkeypatched keys	Any real client construction. A suite that needs an API key is a suite the reviewer cannot run
3	<code>AlertEvent(</code> appears only where the alert is first created	A new <code>AlertEvent</code> constructed anywhere in the render or envelope path — that is invariant I1 breaking, and it is exactly the bug the two-reroute test exists to catch
4	API failure falls back to the template <i>and</i> writes an audit row	A silent <code>except: pass</code> — you would be unable to explain your own output on camera

VERIFY — RUN IT YOURSELF, THEN COMMIT

```
pytest tests/ -v                                # with a key present
OPENAI_API_KEY= pytest tests/ -v                # and with it absent — both must pass
git check-ignore -v .env                        # must print the ignore rule
git status                                       # .env must NOT appear
git commit -am "feat(module-5): context synthesis, explanation layer — Gate 5 green"
```

EXPLAIN IT BACK — OUT LOUD, WITHOUT LOOKING

1 • What does the LLM actually do here?

"It writes prose about a decision that has already been made. It gets no tools, no candidate list and no authority to change the recipient, the channel or the role. There is a deterministic template fallback, so the whole system runs identically with no API key — which is also why the tests never touch the network."

2 • What does "full context preserved" mean concretely?

"The final envelope carries the original alert object itself, not a reconstruction, plus the complete audit trail across every re-route, plus a numeric comparison against the runner-up. The test drives two consecutive re-routes and asserts the alert is unchanged at the end."

3 • Is there a prompt-injection risk?

"In principle — metric names flow into the model's context. The blast radius is nil by construction: the model has no tools, no routing authority, and its output is prose shown to a human. It is worth naming rather than pretending it does not exist."

GATE 5

- ☐ All four diff checks pass
- ☐ You can give the three answers above unaided
- ☐ Suite green both with and without `OPENAI_API_KEY`
- ☐ No test performs a network call
- ☐ The original alert survives two re-routes by identity, not by resemblance
- ☐ Every passed-over person carries a numeric reason
- ☐ `.env` is ignored and unstaged — verified, not assumed

CLI, HTTP Surface & the Invariant Suite

What the reviewer actually runs and what the camera actually records. Everything before this was for this.

PRE-FLIGHT

- ☐ Gates 1–5 all green and committed
- ☐ `rm -f alert_router.db*`
- ☐ Appendix A open — the four scenarios are the specification for this module
- ☐ Screen recorder installed and tested (OBS, QuickTime, or Loom). Record thirty seconds of your terminal now and play it back; discovering your font is unreadable at 1080p on Monday afternoon is avoidable today

ADDENDUM — READ BEFORE PASTING

#	THE TRAP	WHY, AND WHAT TO DO
1	<code>--scenario all</code> colliding with itself	Four scenarios in one process reuse the database. Alert ids, idempotency keys and evaluation rows all collide, and the second scenario fails on constraints that are working exactly as designed. Each scenario needs a fresh alert id <i>and</i> a clean slate — reinitialise between runs and say so in the output.
2	A demo that only runs on your machine	The reviewer clones and runs. Test that literally: clone into a temp directory, fresh venv, <code>pip install -r requirements.txt</code> , <code>--scenario all</code> with no API key. Anything that depends on your shell history, your <code>.env</code> , or a database file already sitting there is a failure you will not see until it is too late.
3	Output that does not survive compression	The video is watched at whatever resolution the reviewer has. Rich panels look wonderful at native retina and turn to mush at 1080p. Increase the terminal font, keep lines under about 100 characters, and never distinguish anything by colour alone — the suppression line needs a label, not just a shade of grey.
4	Timing-dependent demos	The scenarios inject an interrupt at a moment. Drive that from the executor's phase hook with a seeded RNG, not from wall-clock <code>sleep</code> , or your recording will occasionally show the wrong matrix row firing — on camera, with no way to explain it.
5	An unauthenticated presence endpoint, unmentioned	<code>POST /presence</code> lets anyone flip anyone's availability. Correct for a demo, indefensible in production. Naming it in the README's "what is unfinished" earns credit; leaving the reviewer to find it does the opposite.
6	Writing the README last, quickly	It is read <i>first</i> , and often before the code. The invariant → mechanism → test table is the single highest-value thing in the repository, because it converts "I was careful" into "here is the line that makes it impossible".

THE PROMPT — PASTE THIS INTO CURSOR

Build the demo surfaces and the full invariant suite, per `docs/Master_Vibecoding_Plan_Alert_Routing_Agent.pdf` Appendix A and Appendix C.

1. `alert_router/cli.py` — Typer + Rich: `alertctl demo --scenario [reroute|escalate|floor|failover|all]`. For each scenario show, in order: the alert; the ladder as a table with every score term broken out; dispatch beginning; the interrupt landing; the matrix row that fired with its rationale; any suppression with its numeric reason; and the final envelope. Also `alertctl query-count --alert-id X`, printing

the `evaluations` rows so the query budget is visible on screen.

Each scenario runs against a clean database with a fresh alert id. Keep output legible at 1080p: no line over ~100 characters, and never encode meaning in colour alone.

2. **alert_router/api.py** — FastAPI: `POST /alerts` to trigger routing, `POST /presence` to inject a status change onto the `PresenceBus`, `GET /alerts/{id}/audit` to read the trail. The presence endpoint is deliberately unauthenticated for the demo; note that in the README.
3. **tests/test_invariants.py** — all four invariants asserted end to end **across every scenario**, as a parametrised matrix:
 - **I1** — the final envelope's alert equals the original and the audit trail is complete
 - **I2** — no stakeholder appears twice in `dispatch_attempts` with `state='committed'`, on any channel
 - **I3** — `SELECT count(*) FROM evaluations WHERE alert_id=?` equals the number of distinct people evaluated, and no person appears twice
 - **I4** — no committed recipient has lower `qualification` than someone suppressed for being under-qualified
4. **README.md** per Appendix C. Lead with the problem in one paragraph, then the four invariants as a table of **invariant** → **mechanism** → **test name**. Then run instructions, the R1–R11 table reproduced in full, design decisions you would defend, and an honest, specific "What is unfinished" section.
5. **.env.example** with an empty `OPENAI_API_KEY=` and a comment stating the demo runs without it.

Gate: the full suite green, and `alertctl demo --scenario all` working from a fresh clone in a clean virtual environment with no API key set. Test that literally — clone to a temporary directory and run it there.

READ THE DIFF — BEFORE YOU RUN ANYTHING

```
# 1 – each scenario must start clean
grep -n "init_db|alert_id|reset" alert_router/cli.py | head -20

# 2 – nothing secret may be tracked
git ls-files | grep -E "\.env$|.db$|.db-wal$|.db-shm$"          # must print nothing

# 3 – the README must lead with the invariant table
head -40 README.md

# 4 – output must survive compression
grep -n "width\|Panel\|Table" alert_router/cli.py | head
```

#	ACCEPT IF	REJECT IF
1	Every scenario gets a fresh alert id and a clean database	Four scenarios sharing one database. Alert ids and idempotency keys collide, and scenario two fails on constraints that are working perfectly
2	Nothing printed	Anything printed. The repository is about to be public
3	Problem statement, then invariant → mechanism → test as a table	Installation instructions first. This table is the highest-value paragraph in the repository and it should be visible without scrolling
4	Lines under about 100 characters; nothing distinguished by colour alone	Wide panels that will turn to mush at 1080p, or a suppression line that is only identifiable by being grey

VERIFY — THE FRESH-CLONE REHEARSAL

```

pytest -v                                # everything, all modules

cd /tmp && rm -rf clonetest
git clone ~/Downloads/lec-alert-routing-agent clonetest && cd clonetest
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
unset OPENAI_API_KEY
python -m alert_router.cli demo --scenario all
pytest -v

```

If anything fails here it will fail for the reviewer. This rehearsal catches the two classic submission killers: a file that was never committed, and a dependency you installed globally months ago and forgot about.

```
git commit -am "feat(module-6): CLI, HTTP API, invariant suite, README – Gate 6 green"
```

EXPLAIN IT BACK — THIS ONE IS THE VIDEO

1 • Summarise the whole system in four sentences.

"An alert arrives and one indexed query returns every candidate in that domain — that query is the entire budget, and the database enforces it. The candidates are scored on two independent axes, competence and reachability, and frozen into a ladder. Dispatch runs as a state machine with an explicit commit point, and presence changes arrive as push events carrying their new state, so the agent can react mid-flight without asking anything twice. An eleven-row decision table then chooses between continuing, failing over, aborting and re-routing, holding and escalating up, or completing and escalating in parallel."

2 • What is the weakest part?

"Crash recovery. A process death between reserved and committed leaves an orphaned row and the alert stalls. I know the fix — age reserved rows out with a sweeper — and I chose to ship four working scenarios instead. It is named in the README."

3 • What would break first at scale?

"The PresenceBus is in-process, so two workers would each hold their own view of who is available. The fix is Redis pub/sub for presence and Postgres with SKIP LOCKED for dispatch, and the interfaces are already shaped for it."

Rehearse these three before you record. The brief is explicit that a person watches the video and that they care whether you can defend every decision — these are the answers that make the difference between a demo and an engineering walkthrough.

GATE 6

- ☐ All four diff checks pass
- ☐ You can give the three answers above unaided
- ☐ Full suite green from a fresh clone, fresh venv, no API key
- ☐ All four scenarios run and each fires its expected matrix row
- ☐ `--scenario all` completes without constraint collisions
- ☐ Output is legible in a test recording at 1080p
- ☐ README leads with invariant → mechanism → test
- ☐ "What is unfinished" is specific, including the unauthenticated presence endpoint

Appendix A — Demo scenarios

These four are the specification for Module 6 and the storyboard for the video. Each one exists to make a different matrix row visible.

SCENARIO	SETUP	INJECTED MID-FLIGHT	EXPECTED
reroute	CRITICAL db_replica_lag_seconds 94s vs 30s, infrastructure. Ladder: Priya L3 on-call (139, online) → Tom L1 on-call (123, online) → Elena L5 (140, offline).	Mid-Slack-send, Priya → offline.	R3. Abort inside the window, row <code>aborted</code> , re-route to Tom ($123 \geq \text{floor } 120$). Zero new queries. Tom's envelope names Priya and the reason.
escalate	Same alert, dispatch to Priya proceeding normally.	Elena (L5, 140) → online.	R9. Priya's send completes as <code>fyi</code> ; Elena paged in parallel as <code>escalation</code> . Two notifications, two people, zero duplicates.
floor	Same alert, but Tom is <i>not</i> on-call — qualification 108.	Priya → offline mid-send.	R4. $108 < \text{floor } 120$, so it refuses to re-route. Priya switched to SMS (persistent); Elena paged on SMS. Tom recorded in <code>suppressed</code> : "qualification $108 < \text{floor } 120$ ". Lead the video with this one.
failover	HIGH warehouse_stock_units 12 vs 50, direction <code>below</code> , logistics.	Slack adapter fails on connect.	R6. Same person, next healthy channel, same idempotency key — a continuation, not a duplicate. Also demonstrates a depletion-direction breach that a naive <code>value > threshold</code> router would drop.

WHY FLOOR IS THE ONE TO LEAD WITH Every submission will show a re-route. Almost none will show a system *refusing* to re-route because the only available person is not good enough. That refusal is the exact failure mode the brief calls out — "*must not escalate to someone who is less qualified just because they are online*" — and showing the arithmetic on screen is the strongest thirty seconds you have.

Appendix B — Three-minute video script

Three minutes maximum; anything past 3:00 is not watched. Rehearse once, record twice, keep the second take. Production value counts for nothing — but being unable to explain your own output counts for a great deal.

TIME	ON SCREEN	SAY
0:00–0:20	README, the invariant table	"Alert routing agent. The hard requirement is detecting an availability change mid-dispatch without re-querying anyone — so the registry has two verbs: a counted pull, and a free push. You cannot notice a change you have forbidden yourself to look for, so the look comes to you."
0:20–0:50	demo --scenario reroute, the ladder table	"Critical replica lag. One query, seven candidates, ranked. Qualification and availability are separate axes — that separation is what stops an online junior outranking an offline director."
0:50–1:30	Dispatch begins, interrupt lands, abort → re-route	"Dispatch to Priya on Slack. Mid-flight she drops offline. Row R3: we are pre-commit and Tom clears the floor, so we abort inside the cancellation window and re-route. Query count unchanged — the event carried her new state."
1:30–2:15	--scenario floor, the suppression line	"The interesting case. Same failure, but the only online alternative is under-qualified. Row R4 refuses to route down — it keeps the incumbent on SMS, which survives being offline, and escalates up instead. The junior is recorded as suppressed with the arithmetic, not skipped silently."
2:15–2:45	Final envelope panel	"Final notification: the full original alert, the whole audit trail, and why this person over the runner-up with real numbers. The LLM renders that prose — it never chooses the recipient."
2:45–3:00	pytest -v summary	"Four invariants, each with a named test. Two of them are database constraints rather than code paths. What is unfinished: crash recovery of in-flight dispatches, and real transports — both in the README."

RECORDING CHECKLIST Terminal font large enough to read at 1080p · notifications and Slack quit · a clean shell with no personal paths visible · .env not open in any editor tab · one rehearsal run so the timing is known · record twice.

Appendix C — README skeleton

A person reads this before they read your code, and sometimes instead of it. The invariant table is the highest-value paragraph in the repository.

Alert Routing Agent

One paragraph: an agent that routes operational alerts to the right stakeholder and handles the case where availability changes WHILE the notification is being sent.

The four invariants

#	Invariant	Mechanism	Test
I1	Context is never lost across a re-route	frozen Pydantic models + append-only audit	<code>test_context_survives_two_reroutes`</code>
I2	Nobody is notified twice	<code>UNIQUE dispatch_attempts.idempotency_key`</code>	<code>test_no_duplicate_across_all_scenarios`</code>
I3	One availability query per person per alert	<code>PRIMARY KEY (alert_id, stakeholder_id)`</code>	<code>test_second_pull_raises`</code>
I4	Never escalate downward	<code>qualification`</code> has no availability term + severity floor	<code>test_offline_director_outranks_online_junior`</code>

Two of these are database constraints, not code paths. A second availability query is not a bug I try to avoid – it is a write the database refuses.

Run it

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m alert_router.cli demo --scenario all      # no API key needed
pytest -v
uvicorn alert_router.api:app --reload              # optional HTTP surface
```

Architecture

Execution-flow diagram, then one short section per invariant naming the MECHANISM and the test. Then: pull vs push, and why "detect a change without re-querying" is satisfiable only with two verbs.

The decision matrix

Reproduce R1–R11 in full. Reviewers read tables.

Design decisions I would defend

Pull vs push · the commit point and where the shield goes · channel persistence and why a presence drop mid-SMS is a non-event · qualification ⊥ availability · why the idempotency key excludes the channel · why not Celery or LangGraph · why the LLM renders prose and never routes.

What is unfinished <- be specific; this section earns credit

Crash recovery of RESERVED rows · acknowledgement and de-escalation · time-zone-aware rotas · real transports · single-process PresenceBus · POST /presence is unauthenticated · no rate limiting on escalation storms.

Next steps with more time

Redis pub/sub for presence across nodes · Postgres with SKIP LOCKED for multi-worker dispatch · a learned ranker over historical response times · OpenTelemetry spans per attempt.

Appendix D — Defence notes

The brief is explicit: *"What we care about is whether you can explain and defend every decision in your walkthrough."* These are the questions worth rehearsing out loud.

LIKELY QUESTION	ANSWER
"How do you know you never query twice?"	It is the primary key of <code>evaluations</code> . A second pull raises <code>IntegrityError</code> . It is not a convention I follow; it is a write the database refuses.
"How do you detect a change without querying?"	The event carries the new state in its payload. <code>patch_from_push</code> updates the cached snapshot from the event, so knowledge refreshes while the pull counter stays still. A pull is a question; a push is an announcement. Only questions are counted.
"Why does a channel failover not count as a duplicate?"	The idempotency key is <code>{alert_id}:{stakeholder_id}</code> and deliberately excludes the channel. The same notification down a different pipe is one notification.
"Why not just re-route to whoever is online?"	That is the failure mode the brief calls out. Qualification contains no availability term, so being online cannot raise anyone's score; row R4 refuses the downgrade and escalates up instead. The <code>floor</code> scenario demonstrates it.
"What happens if the recipient goes offline after you have sent?"	Nothing is retracted, because you cannot unsend a message. Row R5 completes the original and escalates in parallel. The commit point is where the abort window closes, and <code>asyncio.shield()</code> marks it in the code.
"Why is a presence drop during an SMS a non-event?"	Because SMS is persistent — it waits on the device. "Offline" means away from keyboard, not unreachable. Treating those as the same thing causes a whole class of pointless re-routes, so channel persistence is a first-class property.
"Why SQLite and not JSON, when the brief allowed JSON?"	Because two of my four invariants become schema constraints. "My code checks for it" is a weaker claim than "the database refuses it", and the difference costs about twenty lines.
"Why no Celery or LangGraph?"	A broker moves the work to another process and puts the cancellation window out of reach — and that window is the assessment. The decision logic is a deterministic eleven-row truth table, not a reasoning loop; wrapping it in an LLM graph would make it slower, non-deterministic and impossible to test exhaustively.
"What does the LLM actually do?"	It writes prose about a decision that has already been made. It gets no tools, no candidate list and no routing authority, and there is a deterministic template fallback, so the system runs identically without an API key.
"What would break first at scale?"	The in-process <code>PresenceBus</code> — it is single-node. Two workers would each hold their own view. The fix is Redis pub/sub for presence and Postgres with <code>SKIP LOCKED</code> for dispatch; the interfaces are already shaped for it. That is in the README, not hidden.
"What is the weakest part?"	Crash recovery. A process death between <code>reserved</code> and <code>committed</code> leaves an orphaned row and the alert stalls. I know what the fix is — age reserved rows out with a sweeper — and I chose to ship four working scenarios instead. It is named in the README.

Appendix E — Troubleshooting

SYMPTOM	CAUSE	FIX
FOREIGN KEY constraint failed on the first pull	The <code>alerts</code> row does not exist	<code>ensure_alert()</code> is missing or not called before the first <code>evaluations</code> insert — Module 1, addendum 1
A <code>DuplicateQueryError</code> that makes no sense	The except clause is too wide	It is catching an FK or CHECK failure. Discriminate on <code>exc.orig</code> — Module 1, addendum 2
<code>PRAGMA foreign_keys</code> returns 0	Pragma set once instead of per connection	Move it into a <code>connect</code> event listener — §3.1
<code>no such table: stakeholders</code> in tests only	<code>:memory:</code> with a real pool	Each connection has its own empty database. Use <code>tmp_path</code> or <code>StaticPool</code>
pytest hangs with no output at all	An <code>async for</code> over a bus that never terminates	<code>PresenceBus.close()</code> must push a sentinel to every subscriber
Tests pass alone, fail together	A stale database, or shared state between tests	<code>rm -f alert_router.db*</code> ; give each test its own <code>tmp_path</code> engine
database is locked	Concurrent writers without WAL	WAL and <code>busy_timeout</code> in the connect listener; keep transactions short; never share a session across tasks
Seed changes have no effect	<code>init_db()</code> seeds on first run only	<code>rm -f alert_router.db*</code> — the <code>*</code> matters, WAL leaves sidecars
Suite hangs at teardown after Module 3	A swallowed <code>CancelledError</code>	Re-raise unless the abort was deliberate — Module 3, addendum 2
Random <code>InvalidRequestError</code> in the executor	An <code>AsyncSession</code> shared across tasks	One session per task, from the factory
The wrong matrix row fires intermittently	Timing-based interrupt injection	Drive phase transitions from the executor's hook, not <code>sleep</code>
The <code>floor</code> scenario re-routes instead of holding	Someone online clears the floor	Check the seed: no other online person may reach qualification ≥ 120 , and Tom must be off-call at 108
<code>ModuleNotFoundError: alert_router</code>	Lost venv, or missing pytest path config	Re-activate the venv; confirm <code>pythonpath = ["."]</code> in <code>pyproject.toml</code>

Appendix F — Submission checklist

The brief disqualifies a private repository and treats an unopenable video as no video. These are the failure modes that cost you the assessment after the engineering is already done.

The repository

- ☐ Pushed to GitHub, **public**
- ☐ **Opened in a logged-out browser window** and confirmed visible — the brief asks for this explicitly, and a private repo is disqualified rather than penalised
- ☐ `git log` shows one commit per module with gate results — the incremental history is itself evidence
- ☐ No `.env`, no `*.db`, no `.venv/` in the tree (`git ls-files | grep -E "\.env$|\.db$" returns nothing`)
- ☐ README renders correctly on GitHub — tables intact, no broken code fences
- ☐ Fresh-clone rehearsal passed: clone, venv, install, `--scenario all`, `pytest -v`, all with no API key
- ☐ `docs/Master_Vibecoding_Plan_Alert_Routing_Agent.pdf` committed — it shows the design preceded the code

The video

- ☐ **Three minutes maximum.** Past 3:00 is not watched, so the ending must land before it
- ☐ Sharing set so **anyone with the link** can view — Loom, Drive, or unlisted YouTube
- ☐ **Opened in a logged-out or incognito window** and confirmed playable
- ☐ Shows all four required beats: alert triggering · recipient selected and dispatch beginning · the mid-flight detection · the final notification with full context
- ☐ The `floor` scenario is in it — that is the strongest thirty seconds you have
- ☐ Audio audible; terminal legible at 1080p
- ☐ No API keys, personal paths, or private tabs visible anywhere on screen

The reply

- ☐ **One reply** containing both links — the brief asks for exactly this
- ☐ Any question sent as a **separate** reply, not bundled in
- ☐ Both links clicked and verified from the email itself, one final time
- ☐ Sent before end of day **Monday 17 August 2026**

LAST THING The brief says working code with an honest walkthrough of what is unfinished beats a polished demo of something that does not run. If something is broken on Monday, say so in the README and in the video, name the fix, and ship it anyway. The section where you explain what you did not finish is not damage control — it is one of the parts they are reading most closely.